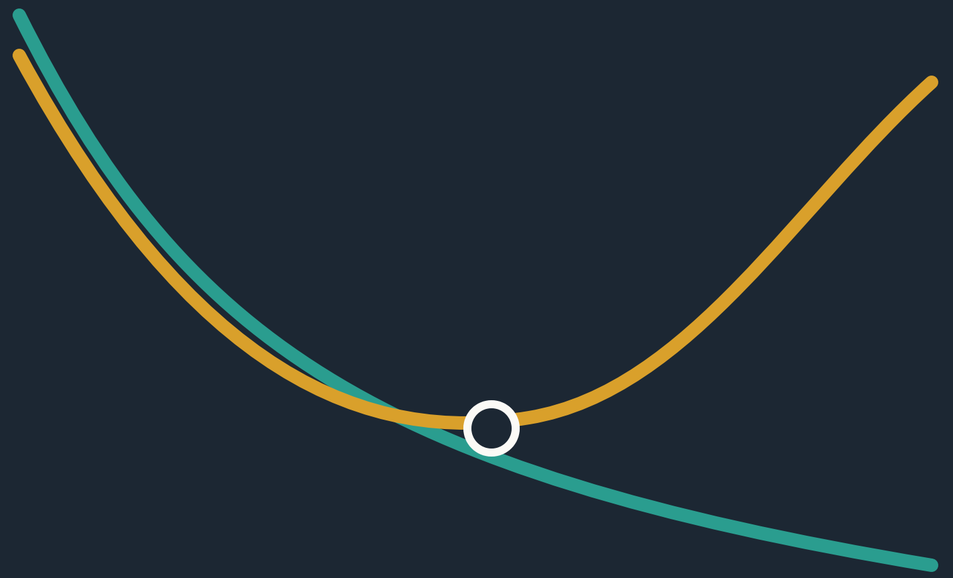
비전 · 딥러닝 — 부록 A

과적합 막기

학습 점수는 100%인데 실전에서 틀리는 이유 — 그것을 재고, 줄이는 법

12시간 · 8차시

실행 코드 10개 · numpy · scikit-learn · PyTorch



부록 A의 지도

재는 법 둘, 막는 법 넷, 종합 하나 + 시작 하나

차시	주제	무엇을 일부러 만들고, 무엇으로 막나	실행
1	과적합을 눈으로	11차 다항식이 점 12개를 전부 지난다	sklearn
2	검증을 제대로	노이즈뿐인 데이터에서 87%가 나온다	sklearn
3	데이터로 막기	더 모으면 나아지나 · 노이즈 열 · 틀린 라벨	sklearn
4	규제	계수를 못 키우게 — Ridge · Lasso	sklearn
5	나무 모델	깊이 · 잎 크기 · 가지치기 · 포레스트	sklearn
6	신경망 ①	조기 종료 · 가중치 감쇠 · 드롭아웃	PyTorch
7	신경망 ② 이미지	배치정규화 · 증강 · 라벨 스무딩 · Mixup · 전이학습	PyTorch·GPU
8	종합	“96.7%” 모델을 넘겨받아 진단하고 고친다	sklearn

기법 색인

어떤 기법이 어디에, 무엇을 하나

기법	차시	종류
교차검증 · 계층 분할	2	재기
누수 막기 · 시험셋 분리	2 · 8	재기
학습 곡선 · 데이터 추가	3	데이터
특징 선택 · 라벨 정리	3	데이터
L2 (Ridge) · L1 (Lasso)	4	규제
깊이 · 잎 크기 · 가지치기	5	규제 (크기)
배깅 · 랜덤포레스트	5	앙상블

기법	차시	종류
조기 종료	6 · 8	규제 (멈춤)
가중치 감쇠	6	규제
드롭아웃	6	규제 · 앙상블
그래디언트 클리핑	6	**안정화**
학습률 스케줄	6	**안정화**
배치정규화	7	안정화 + 약한 규제
증강 · 라벨 스무딩 · Mixup · 전이학습	7	데이터 · 규제

안정화는 학습이 제대로 되게 하는 장치, **규제**는 우연을 못 외우게 하는 장치 — 증상이 비슷해 자주 섞여 불립니다.

시작 전에

이 슬라이드의 숫자는 전부 직접 돌린 값입니다

```
# 콘텐츠 저장소에서
cd deeplearning/appendix-overfit
uv run code/01_see_overfit.py
uv run code/02_validation.py
…
uv run code/08_diagnose.py
→ 표는 화면에, 그림은 figs/ 에
```

uv 하나면 됩니다

파일 맨 위에 필요한 패키지가 적혀 있어 **설치 명령이 따로 없습니다**

시드를 고정했습니다

다시 돌려도 **같은 숫자**가 나옵니다 (7차시 GPU는 소수점 차이 가능)

6·7차시만 PyTorch

7차시는 GPU 권장 — 맥 M 시리즈로 약 10분

1차시 · 90분

과적합을 눈으로 보기

① 학습 오차가 0인데 왜 틀리는가 — 다항식 차수로 직접 만든다

② 학습·검증 오차 곡선에서 갈림길을 찾는다

③ 오차를 편향·분산·노이즈로 쪼개 본다

code/01_see_overfit.py

12장이 남긴 질문

학습 정확도 100%,
그런데 새 데이터에서는 틀린다.
무엇이 잘못됐을까?

모델이 **규칙**을 배운 게 아니라, 이번 데이터에만 있던 **우연**까지 외웠기 때문입니다.

데이터 = 규칙 + 우연

규칙

다음 데이터에도 **똑같이** 있는 것

예: 기온이 오르면 아이스크림이 더 팔린다

우연 (노이즈)

이번 데이터에**만** 있던 것

예: 그날 마침 단체 손님이 왔다

과적합 = 모델이 우연까지 외운 상태.

외운 우연은 다음 데이터에 없으니, 거기서 틀린다.

정답을 아는 문제로 시작합니다

진짜 규칙

$$y = \sin(2\pi x)$$

모델은 이것을 모릅니다. 우리만 압니다

데이터

점 12개

규칙 + 노이즈($\sigma = 0.25$)
검증용은 따로 200개

모델

1차 ~ 11차 다항식

차수 = 복잡도.
11차는 점 12개를 **전부 지날 수** 있습니다

진짜 규칙을 알기 때문에, 모델이 **규칙을 배웠는지 우연을 외웠는지** 눈으로 가릴 수 있습니다.

차수만 바꿔 가며 두 오차를 잰다

```
def sample(n, rng):
    x = rng.uniform(0, 1, n)
    return x, np.sin(2*np.pi*x) + rng.normal(0, 0.25, n)
def fit(degree, x, y):
    # x를 [-1, 1]로 옮겨 풀어 고차에서도 덜 흔들린다
    return Polynomial.fit(x, y, degree)

x_tr, y_tr = sample(12, rng)    # 학습
x_va, y_va = sample(200, rng)   # 검증
for degree in range(1, 12):
    model = fit(degree, x_tr, y_tr)
    train = mse(y_tr, model(x_tr))
    val   = mse(y_va, model(x_va))
```

학습 오차

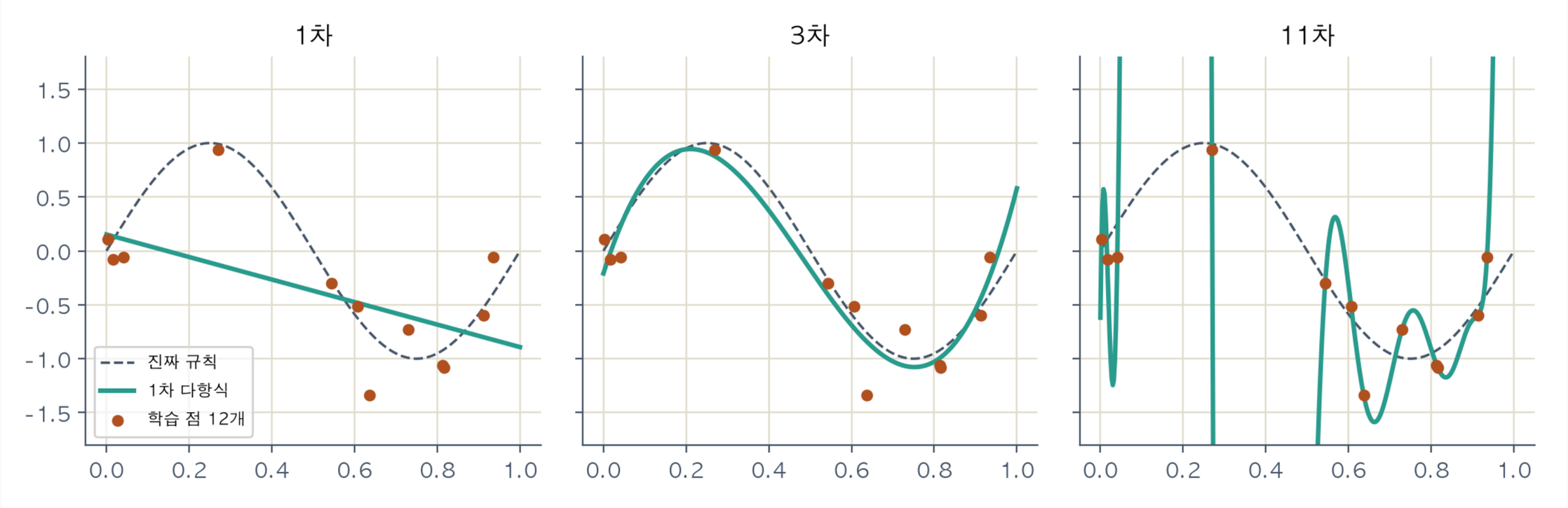
모델이 **본** 점 12개에서의 MSE

검증 오차

모델이 **못 본** 점 200개에서의 MSE.
우리가 진짜 궁금한 것

전체 코드: code/01_see_overfit.py

같은 점 12개, 세 가지 차수



점선 = 진짜 규칙 · 청록 = 모델 · 주황 점 = 학습 데이터 — figs/01_fits.png

너무 단순하거나, 적당하거나, 너무 외우거나

1차

과소적합

직선이라 곡선을 **담을 수가 없다**.
학습 데이터도 못 맞힌다

3차

적당

점을 다 지나진 않지만
진짜 규칙과 거의 겹친다

11차

과적합

점 12개를 **전부** 지난다.
노이즈까지 곡선에 새겼다

학습 데이터를 가장 잘 맞힌 모델(11차)이 **가장 나쁜 예측기**입니다.

1차시 · 결과 ②

학습 오차는 계속 줄고, 검증 오차는 3차에서 바닥

차수	학습 MSE	검증 MSE
1	0.2362	0.4161
2	0.2339	0.4357
3 ◀ 최저	0.0540	0.0940
5	0.0436	0.1045
7	0.0277	1.2019
9	0.0231	0.4384
10	0.0003	183.6
11	0.0000	868.3

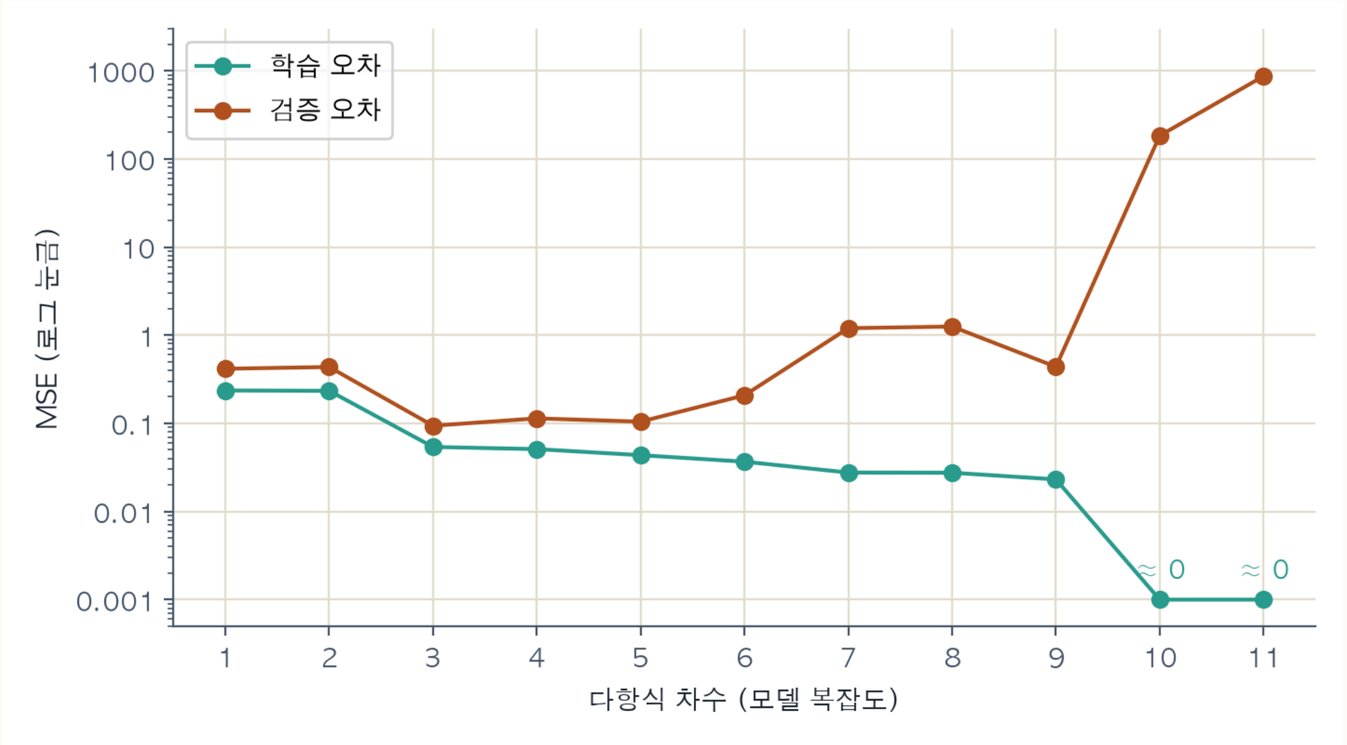
학습 MSE는 **단 한 번도 오르지 않습니다.**
복잡하게 할수록 본 것은 더 잘 맞힙니다

검증 MSE는 3차 **0.094**에서 11차 **868**로 — 약 9,000배

uv run code/01_see_overfit.py 의 [1] 표 (4·6·8차는 생략)

1차시 · 결과 ③

두 곡선이 갈라지는 곳이 과적합의 시작



1 → 3차

검증이 3차에서 크게 내려간다.

아직 배울 **규칙**이 있다

3차 이후

학습만 내려간다.

이제 **우연**을 외운다

세로축은 로그 눈금 · 0.001보다 작은 학습 오차는 “≈ 0”으로 표시 — figs/01_sweep.png

넘을 수 없는 바닥

0.0625

노이즈 자체의 MSE ($\sigma^2 = 0.25^2$)

0.094

가장 좋았던 3차의 검증 MSE

진짜 규칙을 **완벽히** 알아도 검증 오차는 0.0625 밑으로 못 내려갑니다.
학습 오차 0은 목표가 아니라 경고입니다.

두 오차를 함께 봐야 상태를 안다

학습 오차	검증 오차	상태	다음 행동
높다	높다	**과소적합**	모델을 키운다
낮다	낮다 (비슷)	**적당**	여기서 멈춘다
낮다	높다 (벌어짐)	**과적합**	3~7차시의 방법

💡 **원리** — 학습 오차는 “얼마나 외웠나”, 검증 오차는 “얼마나 배웠나”입니다. **하나만 보면 둘을 구별할 수 없습니다.**

데이터를 다시 뽑으면 모델은 어떻게 변할까

편향 bias

여러 번 다시 학습해도 **평균적으로** 정답에서 빗나가는 정도

다트가 늘 **왼쪽 위에 모인다** — 조준이 틀렸다.
너무 단순한 모델(1차)

분산 variance

데이터가 조금 바뀔 때 모델이 **얼마나 흔들리나**

다트가 **사방으로 흩어진다** — 손이 떨린다.
너무 복잡한 모델(11차)

$$\text{기대 오차} = \text{편향}^2 + \text{분산} + \text{노이즈}$$

1차시 · 코드

같은 x에 노이즈만 300번 새로 뽑는다

```
x = np.linspace(0, 1, 12)    # 위치는 고정
points = np.linspace(0.05, 0.95, 50)

for degree in [1, 3, 6, 9]:
    preds = [fit(degree, x, truth(x) + noise())(points)
             for _ in range(300)]
    # 평균 예측이 정답에서 얼마나 먼가
    bias2 = mean((mean(preds) - truth(points))**2)
    # 예측이 판마다 얼마나 흔들리나
    variance = mean(var(preds))
```

현실에서는 데이터를 300번 다시 모을 수 없습니다. **정답을 아는 장난감 문제라서** 가능한 실험입니다

x까지 매번 새로 뽑으면 점이 한쪽에 몰린 판 하나가 평균을 통째로 끌고 갑니다 — 그래서 x는 고정

code/01_see_overfit.py — bias_variance() 를 줄여 옮김

편향은 줄고 분산은 는다 — 합은 3차에서 최소

차수	편향²	분산	노이즈	합 (기대 오차)
1	0.2047	0.0087	0.0625	0.2759
3	0.0067	0.0165	0.0625	**0.0858**
6	0.0003	0.0304	0.0625	0.0932
9	0.0003	0.0634	0.0625	0.1261

편향² 0.2047 → 0.0003
복잡할수록 평균은 정답에 붙는다

분산 0.0087 → 0.0634 (7배)
복잡할수록 판마다 흔들린다

uv run code/01_see_overfit.py 의 [2] 표

과적합을 막는 법은 전부 “분산 줄이기”입니다

차시	방법	분산을 어떻게 줄이나
3	데이터 늘리기 · 정리	우연이 평균에 묻히게 한다
4	규제	모델이 크게 휘지 못하게 묶는다
5	가지치기 · 앙상블	여러 모델의 흔들림을 평균 낸다
6·7	조기 종료 · 드롭아웃 · 증강	외우기 전에 멈추고, 외우기 어렵게 한다

규제·크기 제한의 대가는 편향입니다. 너무 세게 막으면 **1차 직선**으로 돌아갑니다 — 그래서 어디까지 막을지를 **검증으로 정합니다**.

“검증 오차가 더 크면 과적합이다”

⚠ **반만 맞습니다.** 1차 직선을 다시 보세요.

학습 `0.2362` · 검증 `0.4161` — 검증이 `1.8`배

그런데 1차는 **과소적합**입니다. 학습 오차부터 이미 높습니다.

💡 **볼 것은 간격 하나가 아니라 둘의 수준과 추세** — 복잡도를 올렸을 때 **학습은 내려가는데 검증이 오르기 시작하면** 과적합입니다.

“학습 데이터에서 완벽하니 좋은 모델이다”

11차 · 학습 MSE

0.0000

11차 · 검증 MSE

868.3

💡 학습 점수는 **모델이 얼마나 외울 수 있는지**만 알려 줍니다. 쓸 만한지는 **본 적 없는 데이터**로만 알 수 있습니다.

1차시 · 실습 20분

점이 많아지면 11차도 얌전해질까?

1. N_TRAIN을 **12 → 30 → 100**으로 바꿔 세 번 돌린다

2. 매번 표에 적는다 — 11차의 학습·검증 MSE, 검증이 가장 낮은 차수

3. **돌리기 전에** 결과를 먼저 예상해 적는다

N_TRAIN	11차 학습	11차 검증	최적 차수
12	0.0000	868.3	3
30			
100			

MAX_DEGREE는 N_TRAIN - 1로 정해져 있으니 **11로 고정**하고 돌리세요. 이 질문의 답이 3차시입니다.

code/01_see_overfit.py 맨 위 상수

1차시 · 확인 퀴즈

O 일까 X 일까

Q1 다항식 차수를 올리면 학습 오차는 절대 오르지 않는다

Q2 좋은 모델이라면 **기대** 검증 오차를 노이즈 분산(0.0625)보다 낮출 수 있다

Q3 과적합된 모델은 분산이 크다 — 데이터가 조금만 바뀌어도 크게 달라진다

Q4 검증 오차가 학습 오차보다 크면 언제나 과적합이다

1차시 정리

세 줄로

1 과적합 = 규칙에 더해 **우연까지 외운** 상태. 학습 오차 0은 경고다

2 학습은 내려가는데 **검증이 오르기 시작하는 곳**이 갈림길

3 오차 = 편향^2 + 분산 + 노이즈. 막는 법은 전부 **분산 줄이기**

다음 차시 — 그런데 우리가 믿은 “검증 오차”는 믿을 만한 숫자일까?

2차시 · 90분

검증을 제대로 하기

① 같은 모델인데 나누는 운에 따라 점수가 5%p 흔들린다

② 노이즈뿐인 데이터에서 87%가 나오는 순서의 실수

③ 시험지는 맨 처음 떼어 서랍에 넣는다

code/02_validation.py

1차시가 남긴 질문

1차시 내내 검증 오차를 심판으로 썼습니다. 그 심판은 속지 않을까?

과적합을 **막았는지**는 검증 점수로 판정합니다. 검증 점수가 부풀면, 막지 못했는데 막았다고 믿게 됩니다.

2차시 · 개관

검증 숫자가 거짓말하는 다섯 가지

① 나누는 운
한 번 나눈 검증 점수는 흔들린다

② 비율이 깨진 분할
희귀한 클래스가 한 폴드에 0개

③ 누수
검증 데이터를 몰래 본 전처리 — **가장 위험**

④ 시간을 섞은 분할
미래로 과거를 맞히는 시험

⑤ 검증셋에 과적합
많이 시험해 1등을 고르면 운이 섞인다

다섯 가지 전부 **일부러 만들어** 숫자로 보입니다

uv run code/02_validation.py — 실험 [1]~[5]

2차시 · ① 나누는 운

같은 데이터, 같은 모델, 나누기만 50번

```
x, y = load_breast_cancer(return_X_y=True)
scores = []
for seed in range(50):
    # 20%를 검증으로 — seed만 다르다
    x_tr, x_va, y_tr, y_va = train_test_split(
        x, y, test_size=0.2, random_state=seed)
    scores.append(logistic().fit(x_tr, y_tr).score(x_va, y_va))

# 비교: 5-fold 교차검증
cv = cross_val_score(logistic(), x, y,
    cv=StratifiedKFold(5, shuffle=True, random_state=0))
```

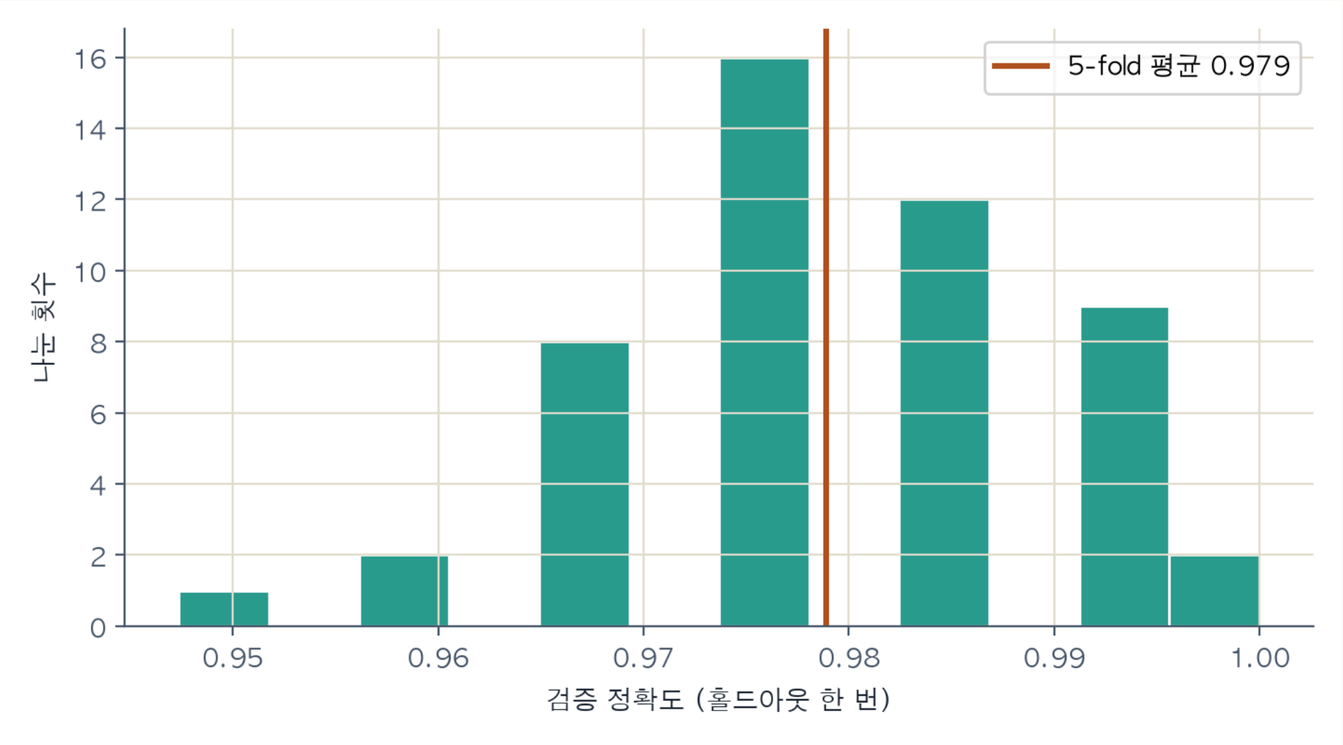
유방암 진단 데이터
569명 · 특징 30개 · sklearn 내장

검증 114명
한 명을 맞히고 틀리는 차이가 **0.9%p**

code/02_validation.py — holdout_luck()

2차시 · ① 결과

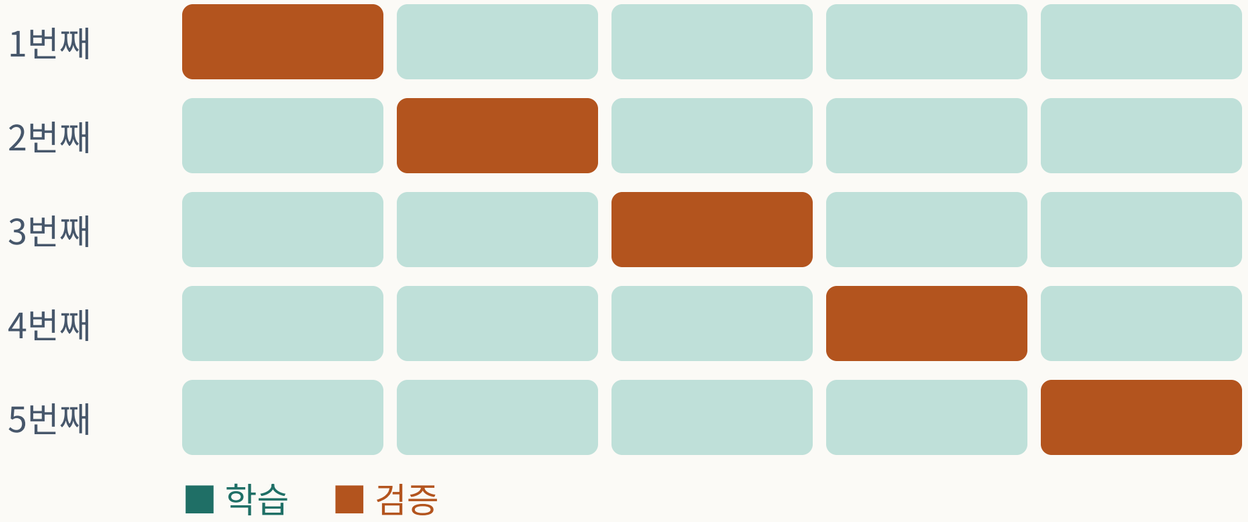
94.7%에서 100%까지 — 운만으로 5.3%p



	홀드아웃	5-fold
평균	0.977	0.979
최저	0.947	0.956
최고	1.000	1.000
표준편차	0.011	0.014

홀드아웃 = 50번 나눈 결과 · 5-fold = 폴드 다섯 개의 점수 — figs/02_holdout_luck.png

모든 데이터가 한 번씩 검증이 된다



데이터를 다섯 조각으로 나눠 **돌아가며** 한 조각씩 검증으로 쓴다

점수 다섯 개의 **평균**을 보고, **흩어짐**도 함께 본다

대가: 학습을 5번 한다

2차시 · ② 비율이 깨진 분할

200개 중 양성이 10개라면

분할	폴드마다 검증에 든 양성	최소 ~ 최대
KFold	0 · 2 · 3 · 4 · 1	**0** ~ 4
StratifiedKFold	2 · 2 · 2 · 2 · 2	2 ~ 2

⚠ 첫 폴드는 양성이 **0개** — “양성을 하나도 못 맞히는 모델”도 이 폴드에서는 만점입니다.

💡 분류에서는 **계층 분할**(stratify)이 기본값. `train_test_split(…, stratify=y)`

uv run code/02_validation.py 의 [2] 표

2차시 · ③ 누수

맞힐 것이 하나도 없는 데이터를 만듭니다

샘플

100

명(환자·제품…)

특징

5,000

개 — 전부 난수

라벨



동전 던지기

특징과 라벨 사이에 관계가 **전혀 없습니다**.
정직한 검증이라면 정답은 **50%**여야 합니다.

유전자 발현·센서 로그처럼 “샘플은 적고 열은 많은” 데이터를 흉내 냅니다

2차시 · ③ 누수 — 코드

같은 두 단계, 순서만 다르다

✗ 먼저 고르고, 교차검증

```
# 전체 100명의 라벨로 20개를 고른다
x_sel = SelectKBest(f_classif, k=20)\
    .fit_transform(x, y)

# 그다음 교차검증
cross_val_score(LogisticRegression(max_iter=2000),
    x_sel, y, cv=cv)
```

✓ 교차검증 안에서 고르기

```
# 고르기를 파이프라인에 넣는다
model = make_pipeline(
    SelectKBest(f_classif, k=20),
    LogisticRegression(max_iter=2000))

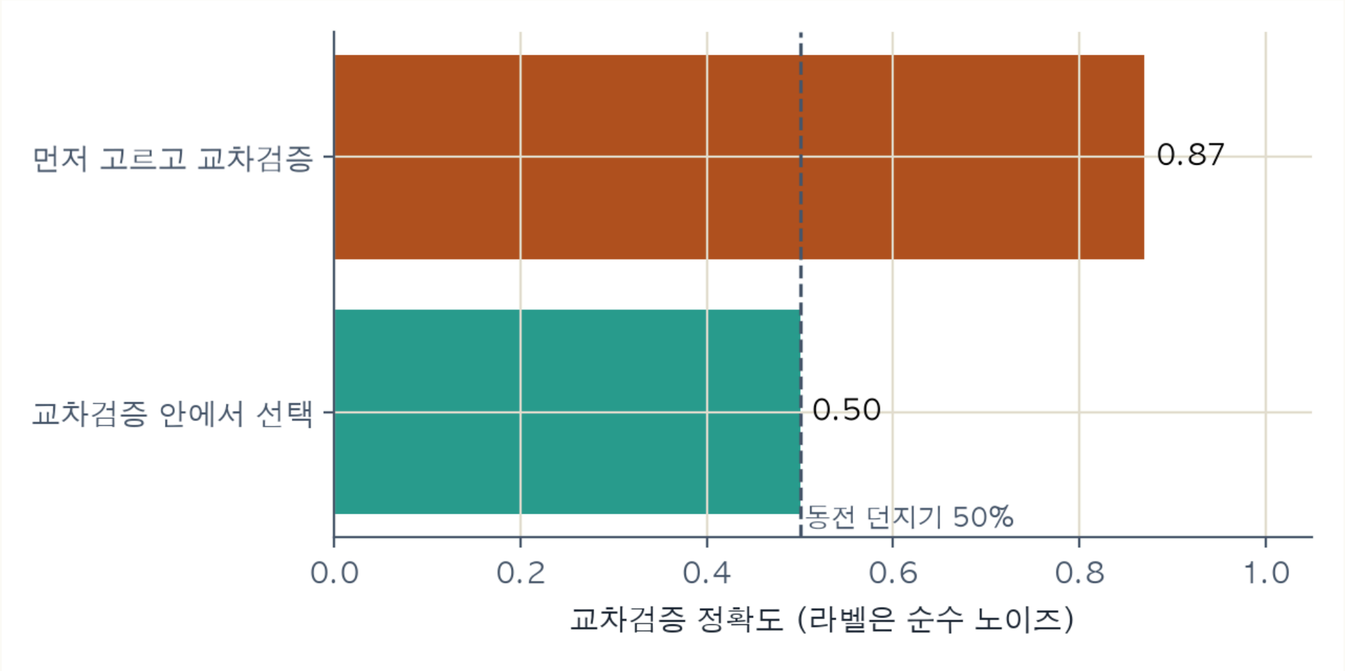
# 폴드마다 학습 쪽 라벨로만 고른다
cross_val_score(model, x, y, cv=cv)
```

왼쪽은 특징을 고를 때 **나중에 검증 폴드가 될 사람들의 라벨까지** 이미 봤습니다.

code/02_validation.py — leakage()

2차시 · ③ 누수 — 결과

동전 던지기를 87% 맞혔습니다



0.87
순서가 틀린 교차검증

0.50
정직한 교차검증 = 정답

uv run code/02_validation.py 의 [3] 표 — figs/02_leakage.png

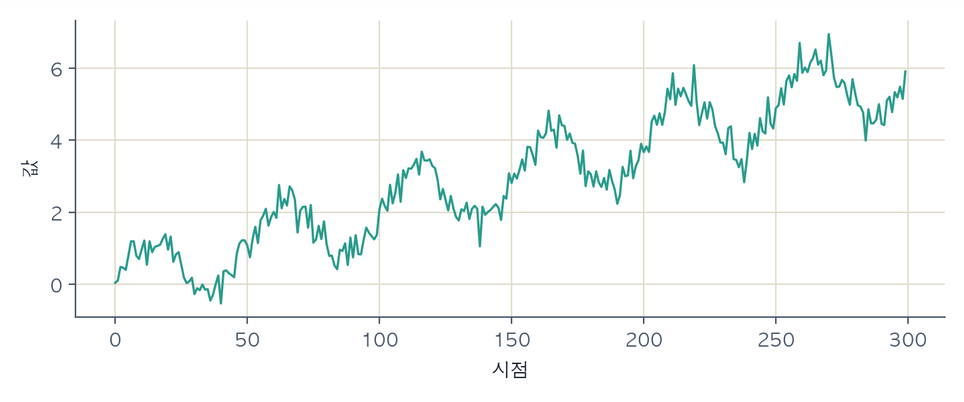
데이터를 보고 정하는 것은 전부 “학습”이다

전처리	무엇을 “배우나”
표준화 (StandardScaler)	평균과 표준편차
결측값 채우기	채울 평균·최빈값
특징 선택 (SelectKBest)	라벨과 상관 높은 열 — **라벨을 본다**
범주 인코딩 · PCA	범주 목록 · 주성분 방향

💡 **규칙 하나** — 이 중 무엇이든 **파이프라인 안에 넣는다.** 그러면 교차검증이 폴드마다 학습 쪽으로만 다시 배운다.

2차시 · ④ 시간을 섞은 분할

섞어서 자르면 오차가 3.6배 작게 나온다



분할 (모델: 가까운 시점 5개 평균)	MAE	무엇을 잰 것
KFold — 섞어서	0.270	사이 값 메우기
TimeSeriesSplit — 과거 → 미래	0.979	진짜 예측

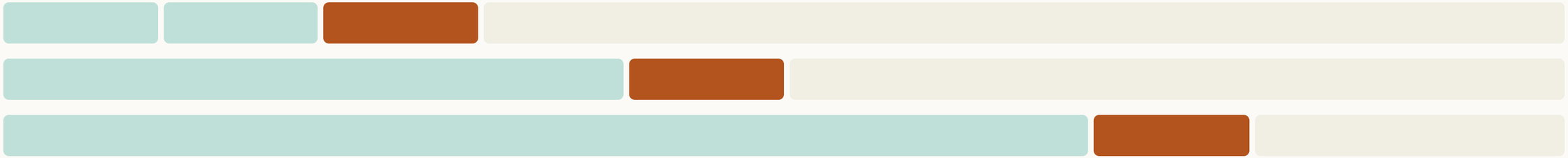
uv run code/02_validation.py 의 [4] 표 — figs/02_timeseries.png

검증은 언제나 학습보다 “미래”여야 한다

KFold(shuffle) — 검증 시점이 학습 시점 사이사이에 끼어 있다



TimeSeriesSplit — 과거로 배우고 바로 다음 구간을 맞힌다



■ 학습 ■ 검증 ■ 아직 쓰지 않음(미래) → 시간

2차시 · ⑤ 검증셋에 과적합

찍기 200개 중 1등을 뽑았더니

후보 200개의 검증 평균

0.50

1등의 검증 점수

0.62

같은 실력, 새 시험

0.48

후보는 전부 **아무것도 모르는 찍기**입니다. 실력 차이는 0인데 1등은 62%가 나왔습니다 — **운이 좋았던 후보를 고른 것**입니다.

하이퍼파라미터 조합 200개를 검증 점수로 비교해 고르는 일이 정확히 이것입니다.

uv run code/02_validation.py 의 [5] 표 · 검증 100문항 · 찍기 정답률 50%

데이터는 세 덩어리 — 시험지는 서랍에

학습
가중치를 배운다

검증
모델·설정을 **고른다**

시험
마지막에 **한 번** 잰다

💡 교차검증을 쓰면 학습+검증 부분을 돌려 쓴다. **시험은 그래도 따로** 뗀다

⚠ 시험 점수를 보고 다시 고치면, 그 순간 시험지는 **검증으로 바뀐다**

상황별로 어떻게 나누나

상황	나누는 법
분류, 데이터 수천 개 이하	`StratifiedKFold` (5~10)
데이터가 아주 많다	홀드아웃 한 번으로 충분 (stratify=y)
시간 순서가 있다	`TimeSeriesSplit` — 과거로 미래를
한 사람·기계의 기록이 여러 줄	`GroupKFold` — 같은 그룹은 한쪽에만
전처리가 있다	전부 `make_pipeline` 안에
설정을 많이 비교해 고른다	시험지를 따로 떼어 두고 마지막에 한 번

2차시 · 함정

“교차검증을 했으니 믿을 수 있다”

⚠ 87%도 교차검증의 결과였습니다. 교차검증은 **그 앞에서 이미 새어 나간 정보**를 막아 주지 않습니다.

💡 **습관 하나** — 점수가 **생각보다 좋으면 축하보다 의심**을 먼저. 라벨을 섞은 뒤 **특징 선택까지 포함한 절차 전체**를 다시 돌려 봅니다. 그래도 높게 나오면 절차 어딘가가 새고 있습니다.

```
y_shuf = np.random.default_rng(0).permutation(y)
# 이미 고른 x_sel 에 하면 안 된다 — 선택을 섞은 라벨로 다시 해야 한다
x_sel = SelectKBest(f_classif, k=20).fit_transform(x, y_shuf)
cross_val_score(LogisticRegression(max_iter=2000), x_sel, y_shuf, cv=cv).mean()
```

2차시 · 실습 20분

누수의 크기를 직접 재 본다

1 LEAK_KEEP을 **5 · 20 · 200**으로 바꿔 누수 점수를 표로. 몇 개를 고를 때 가장 부풀까?

2 SELECT_CANDIDATES를 **10 · 200 · 2000**으로. 1등의 검증 점수는 어떻게 변하나?

3 [3] 누수 실험에 앞 슬라이드의 **라벨 셔플 검사**를 붙여, 틀린 순서가 들키는지 확인

모두 **돌리기 전에 예상**을 먼저 적습니다.

code/02_validation.py 맨 위 상수 · leakage() · selection_overfit()

2차시 · 확인 퀴즈

O 일까 X 일까

Q1 StandardScaler를 전체 데이터에 fit한 뒤 교차검증하는 것도 누수다

Q2 센서 기록을 섞어서 5-fold로 나누면 실전 오차보다 낙관적으로 나오기 쉽다

Q3 시험 점수가 낮아 모델을 고친 뒤 같은 시험지로 다시 재면 여전히 정직한 시험 점수다

Q4 검증 114명짜리 홀드아웃 한 번에서 1%p 오른 것은 의미 있는 개선이라 보기 어렵다

2차시 정리

세 줄로

1 한 번 나눈 점수는 흔들린다 — **교차검증의 평균과 흩어짐**을 함께 본다

2 데이터를 보고 정하는 모든 단계는 **파이프라인 안**. 시간·그룹은 섞지 않는다

3 고르는 데 쓴 점수는 낙관적이다 — **시험지는 서랍에, 마지막에 한 번**

다음 차시 — 이제 믿을 수 있는 자로 잰다. 첫 번째 처방: 데이터

3차시 · 90분

데이터로 막기

① 더 모으면 나아질까 — 학습 곡선으로 미리 안다

② 쓸모없는 열이 섞이면 가까움이 노이즈로 정해진다

③ 틀린 라벨을 외우는 모델과 버티는 모델

code/03_data.py

1차시 실습의 답

같은 11차, 점만 늘렸더니

학습 점	11차 학습 MSE	11차 검증 MSE	최적 차수
12	0.0000	868.3	3
30	0.0270	0.114	4
100	0.0521	0.074	3

> 💡 모델은 그대로인데 과적합이 사라졌습니다. 과적합은 모델 혼자의 성질이 아니라 **모델 대비 데이터의 양**입니다.

01_see_overfit.py 에서 N_TRAIN 만 바꿔 실측 (MAX_DEGREE = 11 고정)

3차시 · 개관

모델을 묶기 전에 데이터부터 본다

① 양

더 모으면 나아지나?

모으기 전에 학습 곡선으로 미리 판단한다

② 열

쓸모없는 특징

관계없는 열은 외울 거리만 늘린다

③ 라벨

틀린 정답

유연한 모델은 틀린 것까지 외운다

데이터 쪽 처방은 비싸지만 **가장 강력합니다** — 본 과목 12장 5절 ①의 결론과 같습니다.

uv run code/03_data.py — 실험 [1]~[3]

3차시 · ① 학습 곡선 — 코드

데이터를 5% → 100%로 늘려 가며 잰다

```
x, y = load_digits(return_X_y=True)  # 손글씨 1,797장

for model in [DecisionTreeClassifier(random_state=0),  # 깊은 트리
              GaussianNB()]:                           # 나이브 베이즈
    sizes, train, val = learning_curve(
        model, x, y, cv=cv(),
        train_sizes=np.linspace(0.05, 1.0, 10))
```

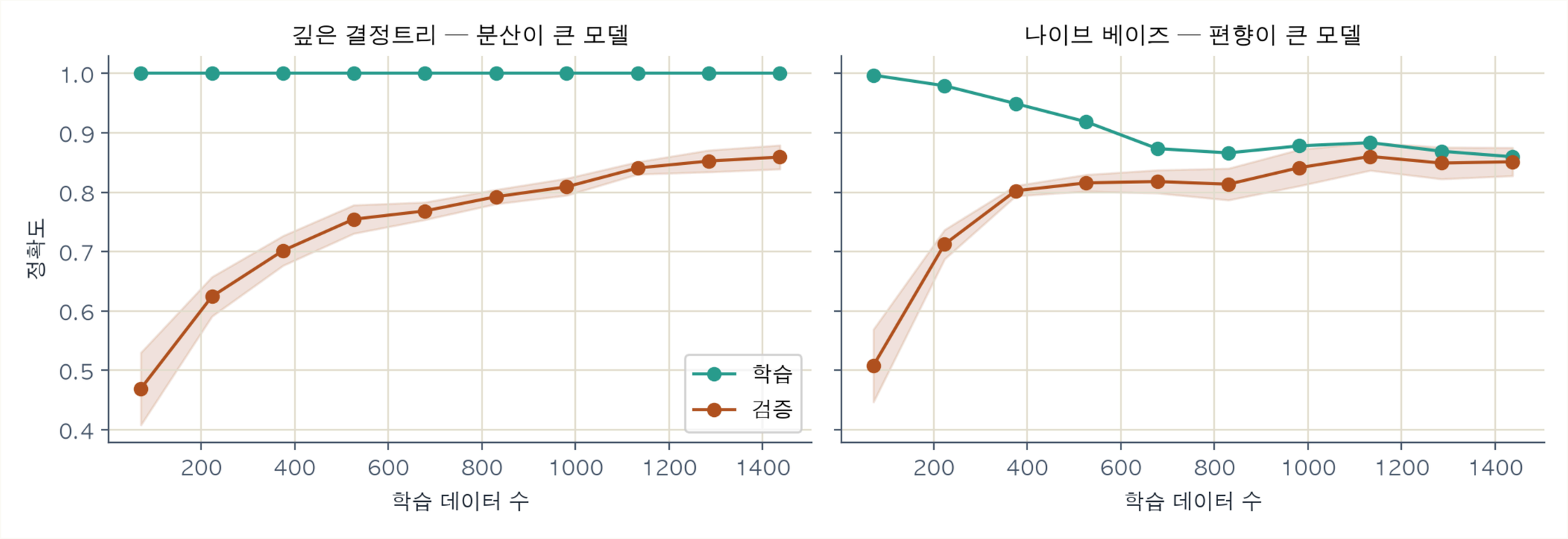
learning_curve
학습 데이터 크기마다 교차검증을 돌려 학습·검증 점수를 돌려준다

왜 두 모델?
곡선 모양이 정반대라, 읽는 법을 대비해서 익힌다

code/03_data.py — curves()

3차시 · ① 학습 곡선 — 결과

벌어진 채로 오르는 곡선, 붙어서 멈춘 곡선



손글씨 숫자 · 5-fold · 띠 = 검증 점수의 ±1 표준편차 — figs/03_learning_curves.png

3차시 · ① 읽는 법

모으기 전에 “모으면 나아질지”를 안다

학습 수	트리 학습	트리 검증	NB 학습	NB 검증
71	1.000	0.469	0.997	0.508
526	1.000	0.755	0.919	0.816
981	1.000	0.809	0.878	0.841
1,437	1.000	**0.859**	0.860	**0.851**

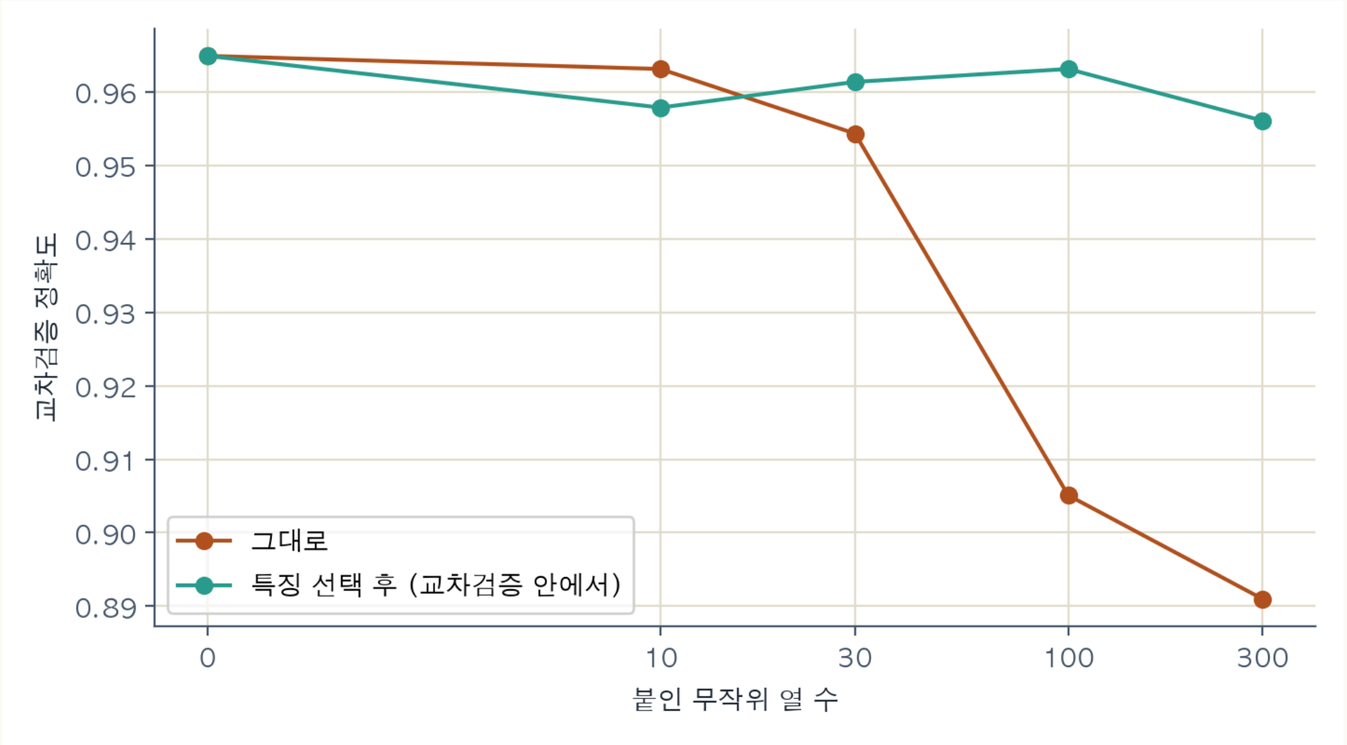
트리 — 간격 14%p, 검증이 아직 오른다 → **더 모은다** 또는 묶는다(4·5차시)

NB — 간격 0.8%p, 평평하다 → 더 모아도 소용없다. **모델을 바꾼다**

uv run code/03_data.py 의 [1] 표 (10단계 중 4단계)

3차시 · ② 쓸모없는 열

무작위 열 300개를 붙였더니 96.5% → 89.1%



붙인 열	그대로	골라서
0	0.965	0.965
10	0.963	0.958
30	0.954	0.961
100	0.905	0.963
300	0.891	0.956

유방암 데이터 진짜 특징 30개 + 무작위 열 · KNN · 고르기는 파이프라인 안(2차시) — figs/03_noise_columns.png

“가까운 이웃”이 노이즈로 정해진다

진짜 열 30개

거리 = 종양 크기·모양이 **비슷한** 환자

+ 무작위 열 300개

거리의 대부분이 **난수 차이** — 이웃이 사실상 아무나

💡 열 하나하나가 모델에게는 **외울 수 있는 우연**입니다. 관계없는 열을 빼는 것은 외울 거리를 줄이는 일입니다.

3차시 · ③ 틀린 라벨

학습 정답의 일부를 틀리게 하면

틀린 라벨	깊은 트리 학습	깊은 트리 검증	얕은 트리 학습	얕은 트리 검증	로지스틱 학습	로지스틱 검증
0%	1.000	0.837	0.882	0.830	0.998	0.972
10%	1.000	0.728	0.806	0.793	0.888	0.906
20%	1.000	0.657	0.730	0.796	0.771	0.891
30%	1.000	0.598	0.639	0.769	0.665	0.846
40%	1.000	**0.487**	0.602	0.656	0.574	**0.852**

손글씨 숫자 · 학습 70% · 검증 30%는 **깨끗한 정답**으로 잰다 · 얕은 트리 = 깊이 8, 잎 최소 5

uv run code/03_data.py 의 [3] 표

3차시 · ③ 읽는 법

유연한 모델일수록 틀린 정답까지 외운다 — 직접 쟀다

틀린 라벨 40%일 때	깊은 트리	얕은 트리	로지스틱
틀린 라벨을 외운 비율	**100%**	21%	**5%**
검증 정확도 (0% → 40%)	0.837 → 0.487	0.830 → 0.656	0.972 → 0.852
검증 하락	-35%p	-17%p	-12%p

💡 외운 비율 = 라벨이 틀린 자리에서 모델이 **그 틀린 라벨을 그대로 내놓은** 비율. 깊은 트리는 하나도 빠짐없이 외웠고, 로지스틱은 거의 따라가지 않았습니다.

uv run code/03_data.py 의 [3-2] 표 · 10~30%에서도 깊은 트리 100%, 로지스틱 4.5~7.2%

데이터 쪽 처방 — 무엇을 먼저 보나

보는 것	신호	처방
학습 곡선	간격 크고 검증이 아직 오름	더 모은다
학습 곡선	둘이 붙어서 평평	모델을 바꾼다 (모으지 않는다)
열	관계없을 열이 많다	빼거나, 파이프라인 안에서 고른다
라벨	손실이 유난히 큰 샘플이 몰려 있다	그 샘플부터 검수한다 · 덜 유연한 모델

비전 과목 19장 — **“라벨이 천장이다.”** 모델은 데이터보다 똑똑해질 수 없습니다.

3차시 · 함정

“데이터는 많을수록 무조건 좋다”

⚠ 두 가지 반례

① 나이브 베이즈 — 981장 **0.841** → 1,437장 **0.851**. 46% 더 모아 1%p

② 틀린 라벨이 섞인 데이터를 더 모으면 **틀린 것도 같이 늘어난다**

💡 더 모으기 전에 **학습 곡선 한 장**. 평평하면 돈 쓰지 말고 모델이나 라벨을 본다.

3차시 · 실습 20분

세 번째 모델의 곡선을 읽어 본다

1 `curves()`에 **로지스틱 회귀**(표준화 포함)를 더하고 `subplots(1, 3)`으로 칸도 늘린다. 곡선은 트리 쪽인가, NB 쪽인가?

2 `noise_columns()`의 KNN을 **RandomForestClassifier**로 바꾼다. 무작위 열 300개에 얼마나 버티나?

3 각 결과로 **처방 한 줄**을 쓴다 — “더 모은다 / 모델을 바꾼다 / 열을 뺀다” 중 무엇이고 근거는?

돌리기 전에 예상부터. 정답은 없고 **근거가 숫자인지**만 봅니다.

code/03_data.py

3차시 · 확인 퀴즈

O 일까 X 일까

Q1 학습·검증 곡선이 붙은 채 평평하면, 데이터를 더 모으는 것이 가장 좋은 처방이다

Q2 라벨 30%가 틀린 데이터에서 학습 정확도 100%는 모델이 틀린 것까지 외웠다는 신호다

Q3 관계없는 열을 추가해도 정보가 늘 뿐이니 성능이 떨어지지는 않는다

Q4 같은 모델이라도 데이터가 많아지면 과적합이 줄어든다

3차시 정리

세 줄로

1 과적합은 **모델 대비 데이터의 양** — 같은 11차도 점 100개면 얌전하다

2 더 모으기 전에 **학습 곡선**: 벌어진 채 오르면 모으고, 붙어서 평평하면 모델을 바꾼다

3 관계없는 열과 틀린 라벨은 **외울 거리**다 — 빼고, 검수한다

다음 차시 — 데이터를 더 모을 수 없다면? 모델이 크게 휘지 못하게 묶는다

4차시 · 90분

규제

① 계수가 크면 벌점 — 11차 다항식을 얌전하게 만든다 (Ridge)

② 쓸모없는 특징의 계수를 정확히 0으로 (Lasso)

③ 얼마나 셀지(α)는 교차검증으로 고른다

code/04_regularization.py

질문

11차 다항식은 점 12개를 지나려고 계수를 약 4,257만까지 키웠습니다.

크게 휘는 곡선에는 큰 계수가 필요합니다. 그렇다면 **계수를 못 키우게 막으면** 덜 휘지 않을까?

4차시 · 원리

손실에 “계수 크기 벌점”을 더한다

손실 = 오차²의 합 + α × 벌점

데이터를 잘 맞혀라 · 그러나 계수를 키우지 마라 · α = 둘 사이의 저울

Ridge (L2)

벌점 = Σ w²

모든 계수를 골고루 **작게**

Lasso (L1)

벌점 = Σ |w|

덜 중요한 계수를 **정확히 0**으로

α = 0 이면 규제 없음 · α가 클수록 계수가 작아진다

4차시 · ① Ridge — 코드

같은 11차, 마지막 한 칸만 바꾼다

```
def poly_model(alpha):
    # ExactFit = 점 12개를 정확히 지나는 해(연립방정식)
    regressor = ExactFit() if alpha == 0 else Ridge(alpha=alpha)
    return make_pipeline(
        PolynomialFeatures(11, include_bias=False),
        StandardScaler(),    # 벌점을 공평하게
        regressor)

for alpha in [0, 1e-6, 1e-4, 1e-2, 1, 100]:
    model = poly_model(alpha).fit(x_tr, y_tr)
```

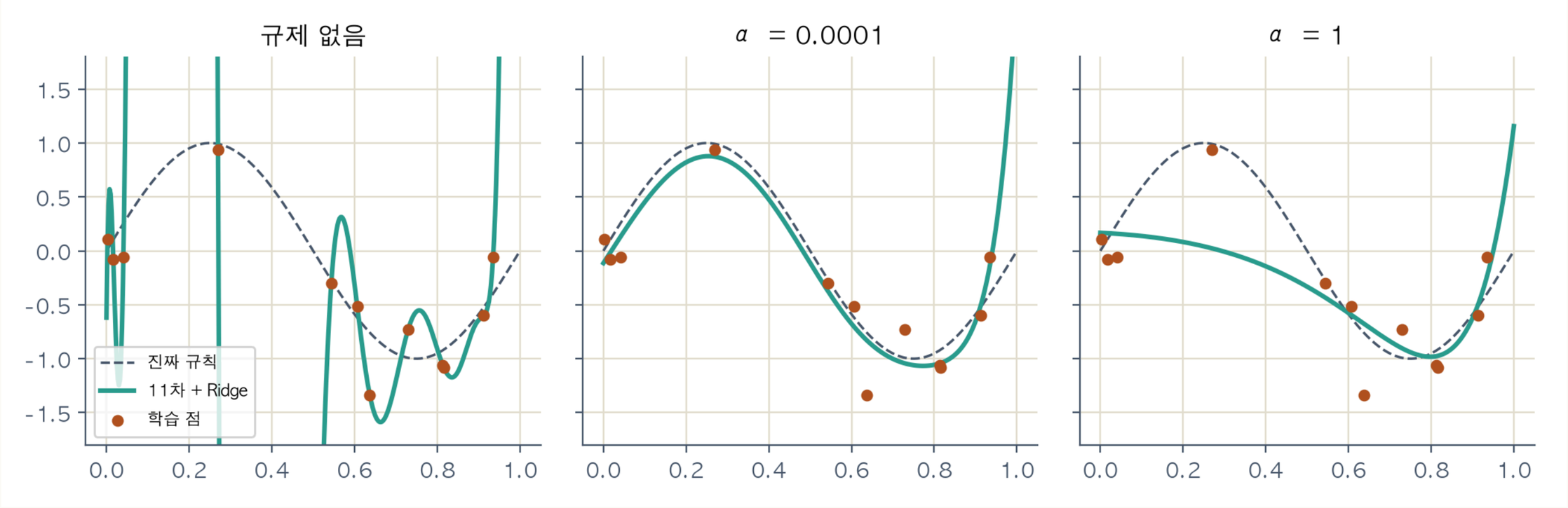
같은 점 12개
1차시와 같은 시드 — 같은 데이터에서 규제만 바꾼다

α를 100배씩
10^{-6} 부터 100 까지, 로그 눈금으로 훑는다

code/04_regularization.py — poly_model(), ridge_sweep()

4차시 · ① Ridge — 결과

너무 약하면 휘고, 적당하면 펴지고, 너무 세면 눕는다



모델은 셋 다 11차 다항식 · 점선 = 진짜 규칙 — figs/04_ridge_fits.png

4차시 · ① Ridge — 숫자

계수가 작아질수록 검증 오차가 내려간다 — 어느 지점까지

α	학습 MSE	검증 MSE	가장 큰 계수
규제 없음	0.0000	868.3	42,566,831
0.000001	0.0287	1.117	42.3
0.0001	0.0416	0.256	11.8
0.01	0.0575	**0.123**	2.53
1	0.1249	0.326	0.30
100	0.3077	0.543	0.03

학습 MSE는 α를 올릴수록 **계속 오릅니다** — 규제는 일부러 덜 맞히게 하는 장치입니다.

uv run code/04_regularization.py 의 [1] 표

4차시 · ② Lasso

특징 50개 중 진짜는 5개 — 누가 골라낼까

학습

60

샘플 (검증 40)

특징

50

개 — 그중 진짜 5개

문제

샘플 60 vs 계수 50

거의 다 맞출 수 있다

```
x, y, true_coef = make_regression(n_samples=100, n_features=50,
    n_informative=5, noise=10, coef=True, random_state=0)
model = Lasso(alpha=alpha, max_iter=50_000).fit(x_tr, y_tr)
kept = np.flatnonzero(model.coef_)    # 0이 아닌 계수의 위치
```

code/04_regularization.py — lasso_sweep()

4차시 · ② Lasso — 결과

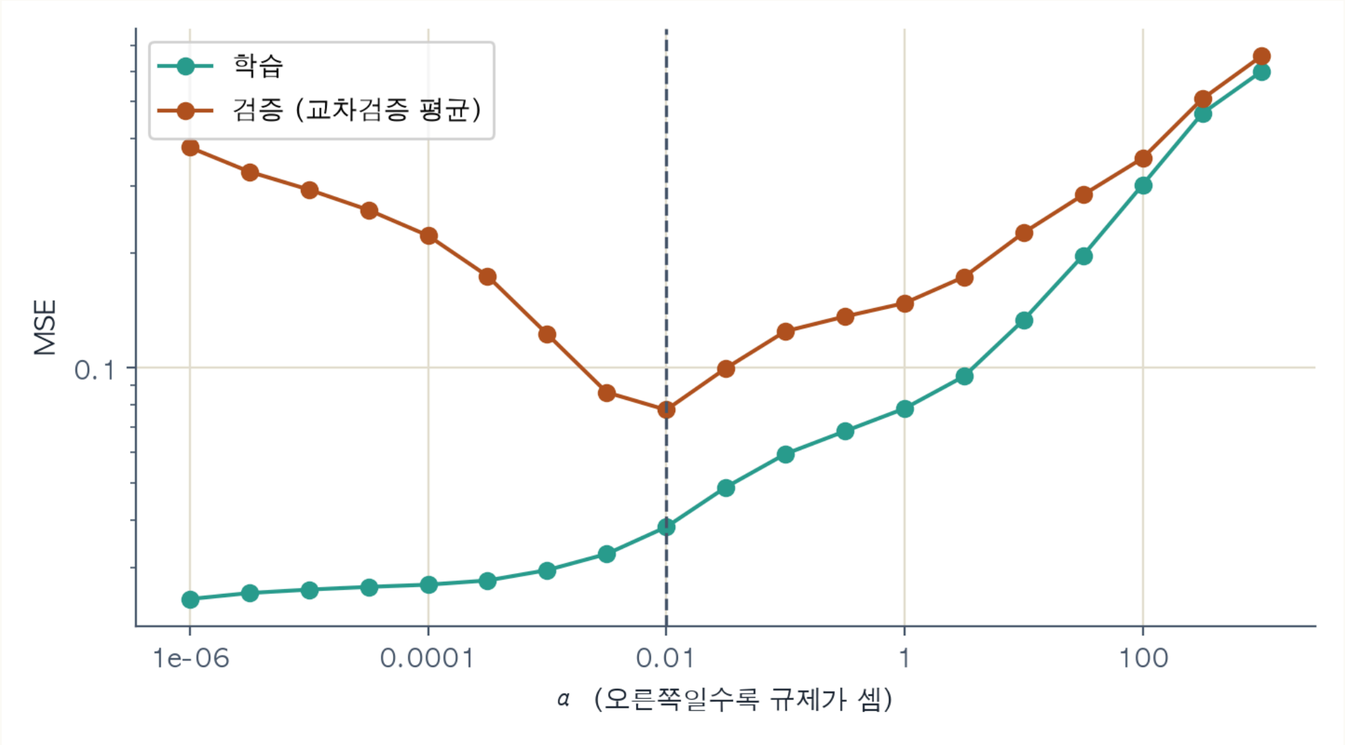
α = 3에서 정확히 진짜 5개만 남았다

α	남은 계수	진짜 중	학습 MSE	검증 MSE
규제 없음	50	-	26.9	359.1
0.01	50	5/5	27.0	317.4
0.1	45	5/5	29.7	237.3
1	18	5/5	65.0	**183.8**
3	**5**	5/5	113.6	194.4
10	5	5/5	407.9	395.0

검증 MSE가 가장 낮은 곳(α = 1)과 정확히 5개만 남는 곳(α = 3)은 **다릅니다**.

uv run code/04_regularization.py 의 [2] 표 (α = 0.001 행 생략)

α도 하이퍼파라미터 — 교차검증으로 고른다




```
validation_curve(
  model, x, y,
  param_name=
    "ridge__alpha",
  param_range=alphas,
  cv=KFold(5, shuffle=True,
    random_state=0))
```

점 15개 · 고른 α = **0.01**

왼쪽 = 과적합 · 오른쪽 = 과소적합 · 점선 = 고른 α — figs/04_validation_curve.png

4차시 · 함정 ①

“α = 0.01이 좋다더라” — 표준화를 했나요?

α	표준화 후 검증 MSE	표준화 없이
0.0001	0.256	**0.101**
0.01	**0.123**	0.273
1	0.326	0.412

⚠ 같은 α가 **전혀 다른 세기**입니다. 최적점이 100배 옮겨 갔습니다. 표준화하지 않은 열은 표준편차가 0.34~0.15로 작아, 같은 효과를 내려면 계수가 커야 하고 그만큼 **벌점을 더 세게** 받습니다 — 고차항일수록 더.

💡 표준화로 벌점을 공평하게 → 그 설정 안에서 **α를 다시 고른다**. 남이 고른 α를 그대로 옮기지 않는다.

같은 점 12개 · 11차 Ridge · 04_regularization.py 의 poly_model 에서 StandardScaler 만 빼고 실측

4차시 · 함정 ②

“규제는 세게 걸수록 안전하다”

Ridge α = 100

학습 0.308 · 검증 0.543
곡선이 **거의 평평**해졌다

Lasso α = 10

학습 407.9 · 검증 395.0
진짜 5개의 계수마저 **깎였다**

💡 너무 센 규제는 과소적합을 만듭니다 — 학습과 검증이 **함께 나빠지면** α를 낮춥니다. 1차시 표의 “높다 · 높다” 칸입니다.

4차시 · 실습 20분

분류에서도 같은 일이 일어나나

1. 유방암 데이터에 `LogisticRegression(C=…)`. **C는 α의 역수** — 작을수록 규제가 세다

2. `validation_curve`로 C = 10^{-4} … 10^{4}를 훑어 U자를 찾는다 (파이프라인에 StandardScaler)

3. `penalty="l1", solver="liblinear"`로 바꿔 특징 30개 중 몇 개가 살아남는지 센다

힌트: `param_name="logisticregression__C"`

04_regularization.py 의 pick_alpha() 를 복사해 시작

O 일까 X 일까

Q1 α를 올리면 학습 오차는 내려간다

Q2 Lasso는 계수를 정확히 0으로 만들 수 있어, 특징 선택처럼 쓸 수 있다

Q3 α는 시험셋 점수가 가장 좋은 값으로 고르는 것이 정석이다

Q4 규제를 쓰면 모델 크기(차수)를 딱 맞게 고르지 않아도 과적합을 줄일 수 있다

4차시 정리

세 줄로

1 규제 = 손실 + **α × 계수 크기 벌점**. 모델은 그대로, 덜 휘게 만든다

2 Ridge는 골고루 작게, Lasso는 **쓸모없는 것을 0으로**

3 α는 **표준화한 파이프라인 + 교차검증**으로, U자 바닥에서 고른다

다음 차시 — 계수가 없는 모델(나무)은 어떻게 묶나

5차시 · 90분

나무 모델

① 멈추라는 말이 없으면 점 하나까지 쪼갠다

② 깊이 · 잎 크기 · 가지치기 — 크기 제한 세 가지

③ 여러 그루를 평균 내면 흔들림이 상쇄된다

code/05_trees.py

5차시 · 트리가 외우는 방식

질문을 계속 던지면 언젠가 점 하나만 남는다

$x_1 < 0.4$?

→ $x_2 < 0.7$?

→ $x_1 < 0.13$?

→ … 잎 하나에 점 하나

반달 데이터

점 1,000개 · 노이즈 0.35 — 두 반달이 꽤 겹친다 · 학습 500 / 검증 500

기본 설정

멈추라는 조건이 없다 → 잎 81개, **학습 정확도 100%**

code/05_trees.py — make_moons(1000, noise=0.35)

5차시 · 크기 제한 ① 깊이

질문을 몇 단계까지만 허락할까 — max_depth

최대 깊이	잎 수	학습	검증
1	2	0.792	0.804
2	4	0.874	0.886
4	12	0.876	0.886
6	25	0.918	**0.890**
8	48	0.954	0.886
12	79	0.994	0.866
제한 없음	81	1.000	0.864

깊이 2~8은 검증이 거의 같습니다(0.886~0.890) — 차이는 검증 500개의 흔들림 안입니다.

uv run code/05_trees.py 의 [1] 표 (깊이 3 행 생략 — 2와 같음)

5차시 · 크기 제한 ② 잎 크기

잎 하나에 최소 몇 개가 남아야 하나 — min_samples_leaf

잎 최소	잎 수	학습	검증
1	81	1.000	0.864
2	70	0.946	0.846
5	44	0.916	0.872
10	30	0.906	0.890
20	17	0.894	**0.892**
50	9	0.824	0.842

20: 학습 0.894 · 검증 0.892 — 간격이 거의 없다

50: 둘 다 내려간다 — 이제는 과소적합

uv run code/05_trees.py 의 [2] 표

5차시 · 크기 제한 ③ 가지치기

다 키운 뒤, 쓸모가 적은 가지부터 자른다

```
# 자를 수 있는 후보 세기를 전부 뽑는다
path = tree().cost_complexity_pruning_path(x_tr, y_tr)
alphas = path.ccp_alphas[:-1]  # 마지막 = 뿌리만

# 후보마다 5-fold — 4차시의 α 고르기와 같다
scores = [cross_val_score(tree(ccp_alpha=a),
          x_tr, y_tr, cv=cv).mean()
          for a in alphas]
```

트리	잎	학습	검증
다 키움	81	1.000	0.864
가지치기	11	0.902	**0.882**

후보 34개 중 교차검증으로 고른 ccp_alpha = 0.0053 · 잎을 **7분의 1**로 줄이고 검증 +1.8%p

uv run code/05_trees.py 의 [3] 표 — pruning()

깊은 트리를 막지 말고, 여러 그루를 평균 낸다

배깅

학습 데이터를 **복원 추출**로 여러 벌 만들어 그루마다 다른 데이터로 키운다

랜덤포레스트

배깅 + 쪼갤 때마다 **특징 일부만** 보게 해 그루끼리 더 다르게

각자는 우연을 외우지만 **외운 우연이 서로 다르다** → 평균에서 상쇄된다.

1차시의 언어로: 편향은 거의 그대로, 분산을 줄인다

5차시 · 앙상블 — 결과

학습은 100%인데 검증이 오른다

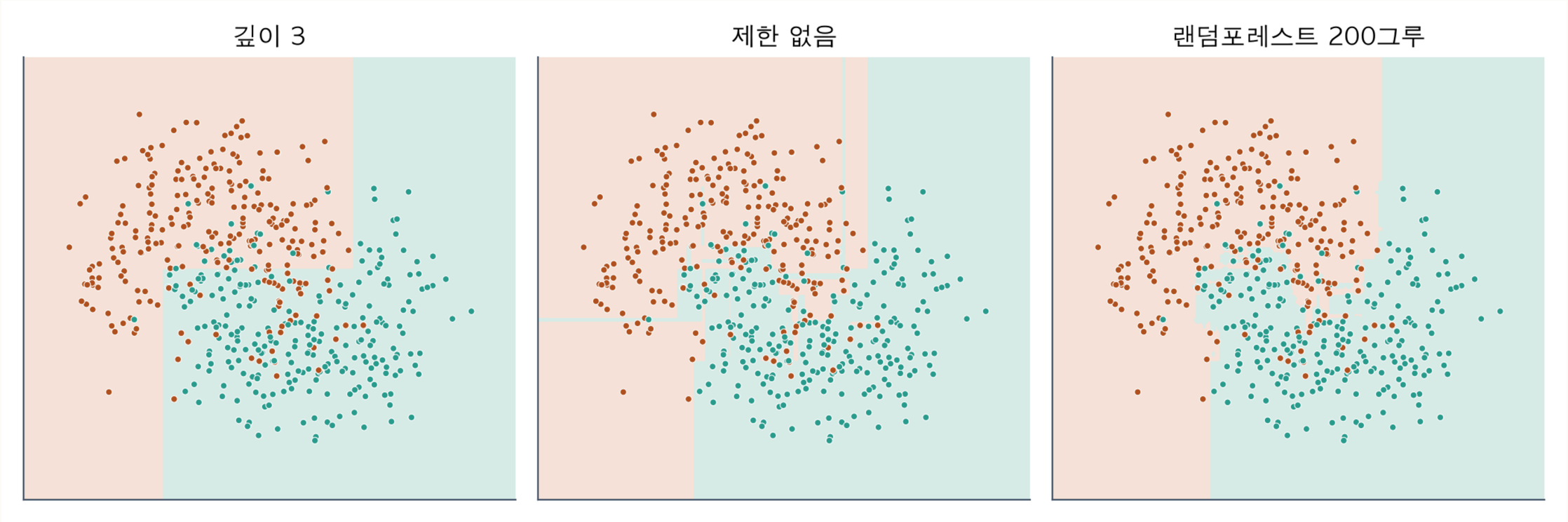
그루 수	배깅 학습	배깅 검증	포레스트 학습	포레스트 검증
1	0.948	0.820	0.912	0.832
5	0.990	0.854	0.988	0.874
10	0.992	0.866	0.992	0.878
50	1.000	0.884	1.000	**0.892**
200	1.000	0.882	1.000	0.884

한 그루 82~83% → 50그루 88~89%. **50 이후는 더 늘려도 그대로**입니다(차이는 흔들림 안).

uv run code/05_trees.py 의 [4] 표 · 그루는 전부 깊이 제한 없음

5차시 · 경계로 보기

외톨이 점을 위한 가는 띠가 사라진다



주황·청록 점 = 학습 데이터 두 클래스 · 바탕색 = 모델의 판정 — figs/05_boundaries.png

5차시 · 함정

“포레스트의 학습 정확도 100% — 과적합이다”

⚠ 학습 점수만으로는 판정할 수 없습니다.

깊은 트리 1그루: 학습 1.000 · 검증 0.864
포레스트 50그루: 학습 1.000 · 검증 **0.892**

💡 학습 점수가 같아도 검증이 다릅니다. 판정은 늘 **검증 곡선**으로 — 1차시 원리 그대로입니다.

덤: 그루마다 복원 추출에서 빠진 약 37%의 점(OOB)으로 검증 없이도 어림할 수 있다 — `RandomForestClassifier(oob_score=True)`

5차시 · 실습 20분

숫자 하나를 믿지 않고 고르기

1. [1] 깊이 표를 홀드아웃 대신 **5-fold 교차검증 평균 ± 표준편차**로 다시 만든다
2. 흩어짐 안에서 비슷한 깊이가 여럿이면 **가장 단순한 것**을 고른다. 어느 깊이인가?
3. `MOON_NOISE`를 0.2와 0.5로 바꿔 포레스트가 한 그루보다 얼마나 이득인지 비교한다

2번 규칙의 이름: **“1 표준오차 규칙”** — 최고점에서 1 표준오차(fold 표준편차 ÷ √K)만큼 내려온 범위 안의 가장 단순한 모델

code/05_trees.py — depth_and_leaf(), data()

5차시 · 확인 퀴즈

O 일까 X 일까

Q1 결정트리의 기본 설정은 학습 데이터를 전부 맞힐 때까지 쪼갠다

Q2 랜덤포레스트는 그루 수를 늘릴수록 과적합이 심해진다

Q3 앙상블은 주로 편향을 줄인다

Q4 min_samples_leaf를 너무 크게 하면 과소적합이 된다

5차시 정리

세 줄로

1 트리는 기본값이면 **점 하나까지** 쪼갠다 — 잎 81개, 학습 100%

2 크기 제한: **깊이 · 잎 크기 · 가지치기** — 셋 다 교차검증으로 고른다

3 또는 막지 말고 **여러 그루를 평균** — 외운 우연이 서로 상쇄된다

다음 차시 — 파라미터 30만 개의 신경망은 무엇을 외우나

6차시 · 90분 · PyTorch

신경망 ①

① 파라미터 30만 개가 틀린 라벨까지 외우는 과정을 본다

② 조기 종료 · 가중치 감쇠 · 드롭아웃을 하나씩 붙인다

③ 하나씩은 약하고, 함께 써야 하는 이유

code/06_nn.py · CPU로 약 10초 · code/06_extras.py

6차시 · 실험 설정

외우기 아주 쉬운 판을 일부러 깐다

학습 데이터

300

장 — 손글씨 숫자
검증 500 (멈춤) · 시험 997 (보고)

모델

301,066

파라미터
64 → 512 → 512 → 10

학습 라벨

20%

틀림 (3차시 ③)
검증 라벨은 깨끗

사진 1장당 파라미터 **1,000개**. 틀린 라벨 60장까지 **전부 외울 수 있습니다**.

code/06_nn.py — N_TRAIN · HIDDEN · LABEL_NOISE

같은 모델에 손잡이 세 개

```
model = nn.Sequential(
    nn.Linear(64, 512), nn.ReLU(), nn.Dropout(dropout),      # ③ 드롭아웃
    nn.Linear(512, 512), nn.ReLU(), nn.Dropout(dropout),
    nn.Linear(512, 10))
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
    weight_decay=weight_decay)                               # ② 가중치 감쇠

for epoch in range(1, 301):
    train_one_epoch(model)
    val_loss, val_acc = evaluate(model, x_va, y_va)     # model.eval() 안에서
    # ① 조기 종료: early_stop 이면, 검증 손실 최저 기록을
    #    20 epoch 동안 못 깰 때 멈춘다 (다음 슬라이드)
```

다섯 가지 설정 · 모두 같은 시드 · 각 300 epoch까지 · 감쇠 0.1 · 드롭아웃 0.5

code/06_nn.py — mlp(), train() 를 줄여 옮김

6차시 · 결과

멈춘 쪽이 이겼다 — 셋 다 쓰면 89.0%

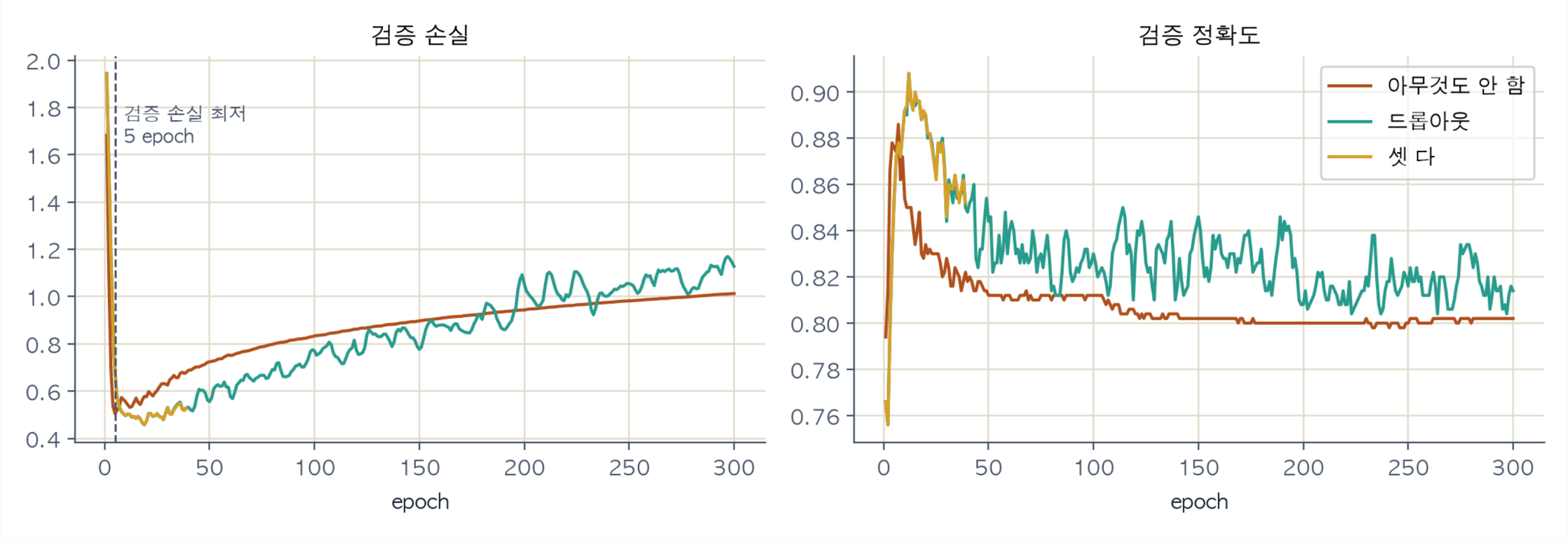
방법	학습한 epoch	학습 정확도	틀린 라벨 외움	시험 정확도
아무것도 안 함	300	1.000	100%	0.829
조기 종료	25	0.797	10%	0.868
가중치 감쇠	300	1.000	100%	0.835
드롭아웃	300	1.000	100%	0.831
셋 다	39	0.867	35%	**0.890**

끝까지 돌린 셋은 틀린 라벨 60장을 **100%** 외웠습니다. 조기 종료는 **6장(10%)**만.

uv run code/06_nn.py · 멈춤은 검증 500장, 점수는 시험 997장 — 1장 = 0.1%p

6차시 · 곡선으로 보기

검증 손실은 5 epoch에 바닥, 그 뒤로는 외우는 시간



주황 = 아무것도 안 함 · 청록 = 드롭아웃 · 노랑 = 셋 다(39 epoch에서 멈춤) — figs/06_curves.png

6차시 · ① 조기 종료

학습 정확도 79.7% — 맞는 라벨이 80%입니다

```
if val_loss < best_loss:
    best_loss, waited = val_loss, 0
    best_state = {k: v.clone() for k, v
        in model.state_dict().items()}
else:
    waited += 1
    if waited ⩾ 20: break   # 인내심 20

# 멈춘 시점이 아니라 가장 좋았던 시점으로
model.load_state_dict(best_state)
```

바닥(5 epoch) + 인내심 20 = **25 epoch**에서 멈춤

맞는 라벨 97.1% · 틀린 라벨은 60장 중 **6장 (10%)**만 외웠다

신경망은 **규칙을 먼저, 우연을 나중에** 배웁니다. 그 사이에서 멈추는 것이 조기 종료입니다.

code/06_nn.py — train() · PATIENCE = 20

4차시 Ridge의 신경망판

매 걸음: w ← w - lr·(Adam 방향) - lr·λ·w

기울기대로 움직이되, 매번 가중치를 0 쪽으로 **조금씩 당긴다** — AdamW가 실제로 하는 일

이번 실험 (λ = 0.1)

시험 정확도 82.9 → 83.5%

틀린 라벨도 여전히 **100%** 외움

PyTorch에서

`AdamW(…, weight_decay=λ)`

Adam보다 AdamW — 감쇠를 기울기와 분리해 건다

code/06_nn.py — WEIGHT_DECAY = 0.1

6차시 · ③ 드롭아웃

매 걸음 뉴런 절반을 무작위로 끈다

학습 중 — `model.train()`

걸음마다 다른 절반이 꺼진다. 특정 뉴런 하나가 **한 장을 통째로 외우는** 일이 어려워진다

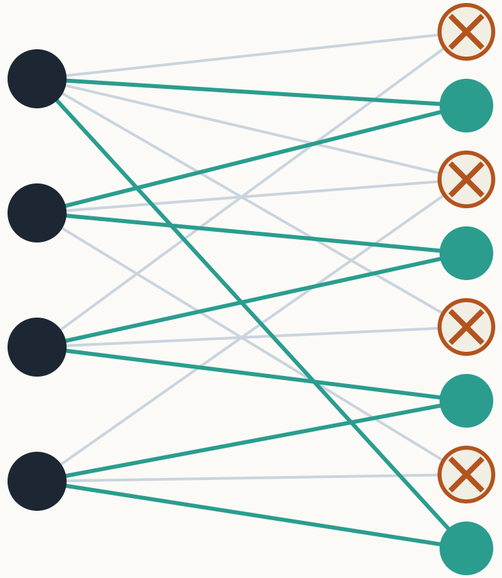
평가 중 — `model.eval()`

전부 켠다. 매 걸음 다른 **작은 망들의 평균**처럼 동작한다 — 5차시 앙상블

이번 실험 (p = 0.5) — 시험 82.9 → 83.1%. 300 epoch를 끝까지 돌리니 드롭아웃이 있어도 **결국 다 외웠습니다**(학습 100%).

code/06_nn.py — DROPOUT = 0.5 · nn.Dropout

같은 입력, 통과할 때마다 다른 뉴런이 꺼진다



뉴런 8개 출력	입력은 전부 1.0 · p = 0.5
train ①	0 0 2 0 0 0 2 2
train ②	0 2 2 2 2 0 2 2
train ③	2 2 0 0 2 0 2 0
eval	1 1 1 1 1 1 1 1

꺼진 칸은 **0**, 남은 칸은 **2배**. 몇 개가 꺼지는지도 매번 다릅니다(5 · 2 · 4개) — 각 칸이 **동전 던지기**로 정해집니다.

uv run code/06_mechanics.py 의 [1] · 왼쪽 그림은 개념도 (× = 이번 걸음에 꺼진 뉴런)

6차시 · ③ 드롭아웃 — 두 가지 질문

왜 2배로 키우나, 왜 과적합을 막나

① 2배 = 평균 맞추기

절반을 끄면 다음 층이 받는 합이 절반이 된다. 남은 칸을 **1/(1−p)**배 하면 평균이 그대로다.

```
train 1만 번 평균 = 0.998
eval 출력        = 1.000
```

그래서 eval 때는 아무것도 안 해도 된다 — 끄기만 하면 끝

② 매 걸음 다른 작은 망

512개 중 무엇이 꺼지느냐에 따라 걸음마다 **다른 부분망**이 학습된다. 가중치는 서로 나눠 쓴다.

eval 때 전부 켜면 그 수많은 부분망을 **평균 낸 것**에 가깝다 — **5차시 배깅**과 같은 원리

한 뉴런에 기대지 못하니, 여러 뉴런이 같은 특징을 나눠 맡게 된다

PyTorch nn.Dropout 은 학습 때 키우는 방식(inverted dropout) — 평가 때 따로 줄이지 않는다

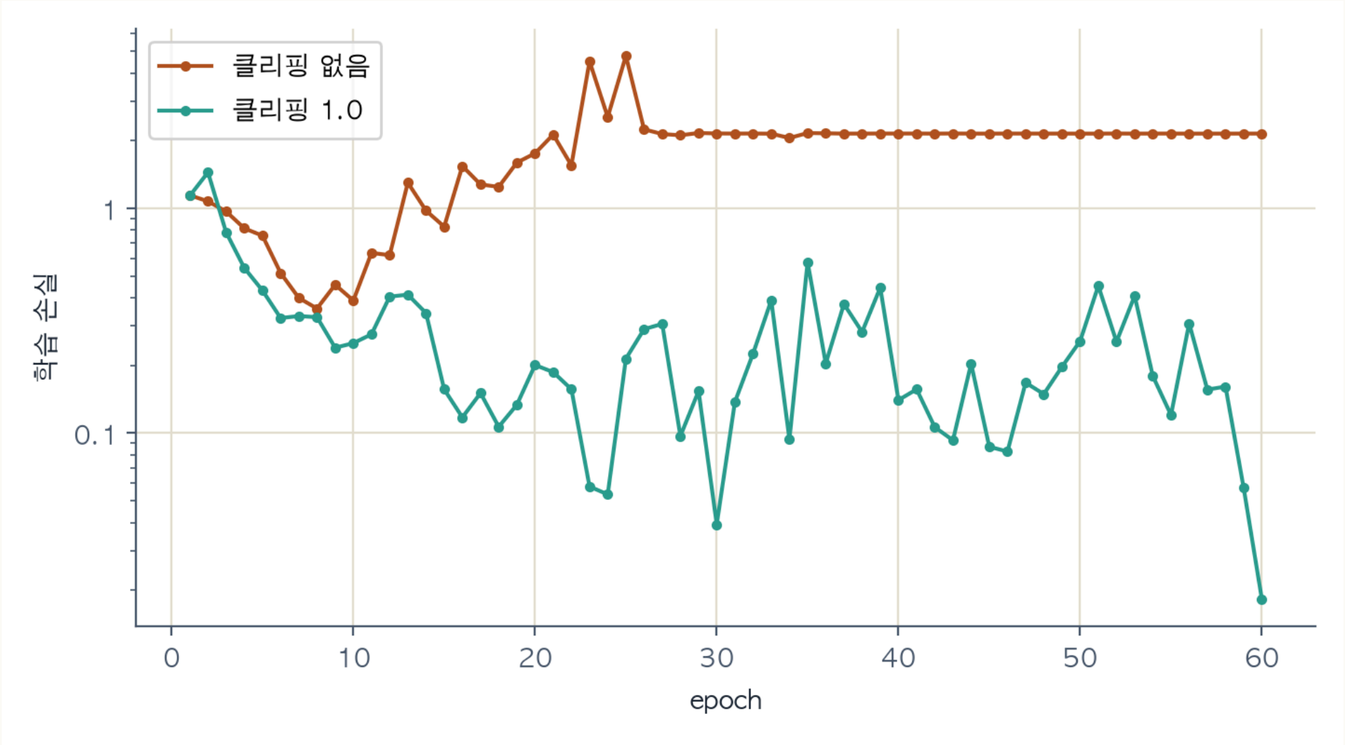
6차시 · 이 실험이 말해 주는 것

감쇠·드롭아웃은 “늦추는” 장치, 멈춤은 “끊는” 장치

감쇠 · 드롭아웃 혼자	외우는 속도를 늦출 뿐 — 300 epoch면 **결국 다 외운다** (+0.2~0.6%p, 흩어짐 안)
조기 종료 혼자	외우기 전에 끊는다 — **+3.9%p**
셋 다	늦춰진 덕에 **더 늦게, 더 좋은 지점**에서 멈춘다 (39 epoch) — **+6.1%p**

모두 “아무것도 안 함” 82.9% 대비 · 시험 997장 · 이 설정(작은 데이터 · 틀린 라벨 20% · 시드 1개)에서

기울기가 너무 길면 잘라 낸다 — 학습이 터지지 않게




```
loss.backward()
nn.utils.clip_grad_norm_(
  model.parameters(),
  max_norm=1.0)
optimizer.step()
```

SGD · 모멘텀 0.9 · lr **0.5** (일부러 크게) · 같은 시드

uv run code/06_extras.py 의 [1] — figs/06_clipping.png · 데이터·모델은 06_nn.py 와 같음

6차시 · 헷갈리기 쉬운 것 ① 읽는 법

클리핑은 발산을 막는다. 과적합은 못 막는다

설정 (60 epoch)	마지막 학습 손실	시험 정확도	상태
클리핑 없음	2.153	0.147	망가짐 (찍기 수준)
클리핑 1.0	0.018	0.800	학습됨 — 그리고 **외움**

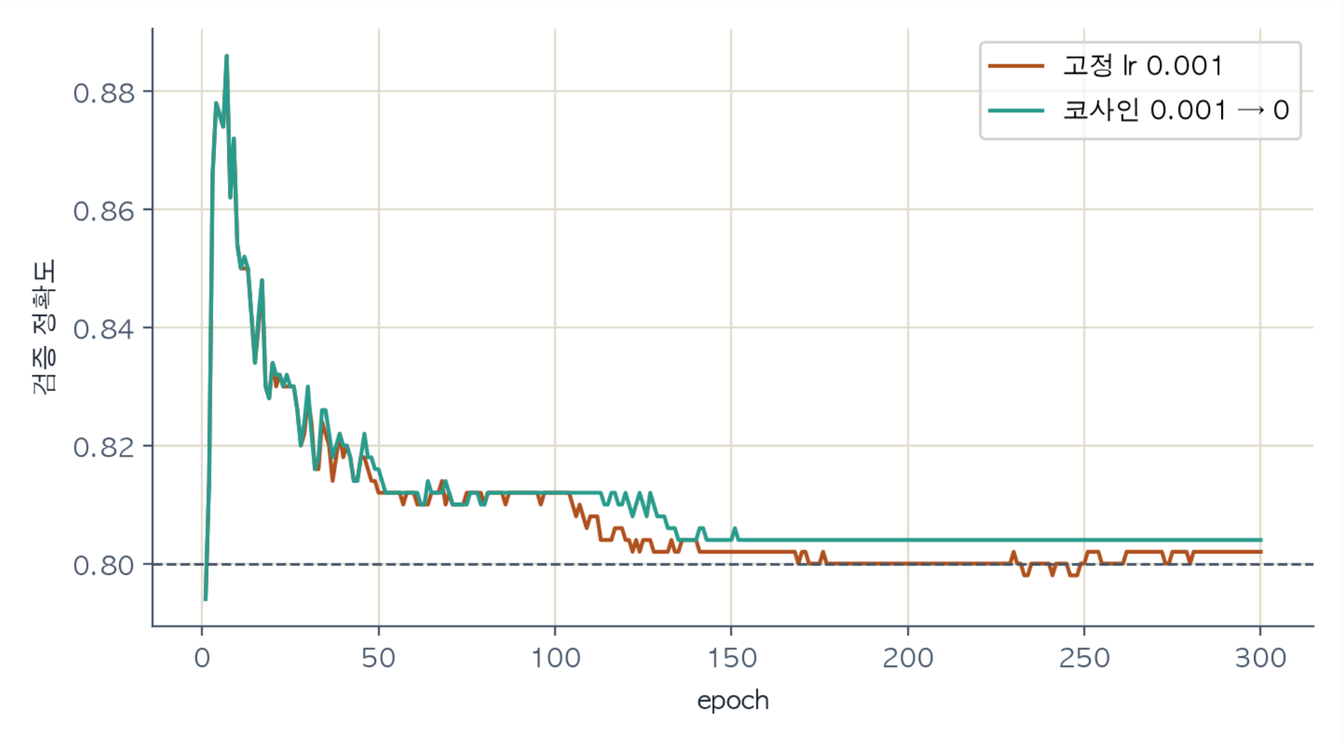
클리핑을 건 모델은 학습 손실 0.018 — 틀린 라벨 60장의 **96.7%를 외웠다**. 시험 80.0%: 클리핑이 한 일은 학습을 살린 것까지다

쓰는 곳: **RNN·트랜스포머**처럼 기울기가 가끔 폭발하는 모델, 큰 학습률로 빨리 학습할 때. 과적합 대책은 따로 필요하다

uv run code/06_extras.py 의 [1] 표 · 찍기 = 10개 중 하나 = 10%

6차시 · 헷갈리기 쉬운 것 ② 학습률 스케줄

학습률을 줄여 가도 결국 외운다



	고정	코사인
검증 최고 (엿봄)	0.886	0.886
그때 epoch	7	7
끝까지 → 시험	0.829	0.833
마지막 학습	1.000	1.000

uv run code/06_extras.py 의 [2] · AdamW · 규제 없음 · 300 epoch — figs/06_schedule.png

6차시 · 함정

평가 때 model.eval()을 빼먹는다

```
# ✗ 학습 모드 그대로
logits = model(x_va)
acc = (logits.argmax(1) == y_va).float().mean()
```

```
# ✓ 드롭아웃 끄고, 기울기도 끄고
model.eval()
with torch.no_grad():
    logits = model(x_va)
```

⚠ 켜 둔 채 재면 검증 점수가 **잴 때마다 달라집니다**. 그 흔들리는 숫자로 조기 종료를 하면 엉뚱한 곳에서 멈춥니다.

💡 그리고 다음 epoch 학습 전에 `model.train()`으로 되돌린다 — 7차시 배치정규화도 같은 스위치

code/06_nn.py — evaluate() 에 @torch.no_grad() 와 model.eval()

6차시 · 실습 20분

손잡이를 직접 돌려 본다

1 `LABEL_NOISE = 0`으로 다시 돌린다. 다섯 방법의 시험 정확도 차이가 커지나, 줄어드나?

2 `PATIENCE`를 5 · 20 · 50으로. 멈춘 epoch, 틀린 라벨 외운 비율, 시험 정확도는?

3 `EPOCHS = 40`으로 줄여 "아무것도 안 함"과 "가중치 감쇠"를 비교하고, `WEIGHT_DECAY`를 0.1 · 1 · 10으로. 감쇠는 정말 약한가, 세기가 약했나, 너무 오래 돌린 탓인가?

CPU 한 번에 약 10초. 예상을 먼저 적고, 표로 비교합니다.

code/06_nn.py 맨 위 상수

O 일까 X 일까

Q1 조기 종료 후에는 멈춘 시점이 아니라 검증이 가장 좋았던 시점의 가중치를 쓴다

Q2 가중치 감쇠는 4차시 Ridge와 같은 발상이다

Q3 드롭아웃은 평가할 때도 켜 두어야 학습 때와 같은 조건이 된다

Q4 이번 실험에서 드롭아웃 하나만으로도 조기 종료만큼 효과가 있었다

6차시 정리

세 줄로

1 신경망은 **규칙을 먼저, 우연을 나중에** 배운다 — 그 사이에서 멈추는 것이 조기 종료

2 가중치 감쇠 ≈ Ridge(SGD에선 같음), 드롭아웃 ≈ 앙상블 — 외우기를 **늦춘다**

3 함께 쓸 때 가장 좋았다: 82.9% → **89.0%**(시험). 평가는 늘 **model.eval()**

다음 차시 — 사진이 적을 때: 사진을 늘리고(증강), 남이 배운 것을 빌린다(전이학습)

7차시 · 90분 · PyTorch · GPU 권장

신경망 ② 이미지

① 배치정규화 — 층마다 숫자의 크기를 맞추는 장치

② 데이터 증강 — 사진 2,000장을 늘려 쓴다 (+ 라벨 스무딩 · Mixup)

③ 전이학습 — 남이 100만 장으로 배운 것을 빌린다

code/06_mechanics.py · code/07_images.py

7차시 · ① 배치정규화 — 무엇을 하나

배치 안에서 특징마다 평균 0 · 분산 1로 맞추고, 다시 늘리고 옮긴다

1

배치 통계를 잰다

μ = 배치 평균

σ² = 배치 분산

→

2

표준화

x̂ = (x − μ) / √(σ² + ε)

→

3

늘리고 옮긴다

y = γ·x̂ + β

γ·β는 학습한다

층을 지날 때마다 숫자의 크기가 들쭉날쭉해지는 것을 **층마다 다시 맞춘다** — 2차시의 StandardScaler를 망 안에, 층마다 넣은 것

nn.BatchNorm1d (완전연결) · nn.BatchNorm2d (합성곱: 채널마다) · ε = 1e-5

크기가 제각각인 네 특징이 한 줄로 선다 — 하나만 빼고

배치 64개	특징 1	특징 2	특징 3	특징 4
입력 평균	4.82	-3.01	104.95	0.01
입력 표준편차	1.73	1.07	27.33	0.001
출력 평균	0.00	0.00	0.00	0.00
출력 표준편차	1.01	1.01	1.01	**0.28**

특징 4는 왜 0.28? 분산 10^{-6} 이 분모의 안전장치 $\varepsilon = 10^{-5}$ 보다 작아서, $\sqrt{(\sigma^2 + \varepsilon)}$를 ε 이 차지했습니다. ε 은 **0으로 나누기를 막는 장치**입니다.

uv run code/06_mechanics.py 의 [2] · train 모드 · $\gamma = 1$, $\beta = 0$ (초기값)

7차시 · ① 배치정규화 — 학습 때와 평가 때

평가 때는 배치가 아니라, 학습 중 쌓아 둔 평균을 쓴다

`model.train()`

이번 **배치**의 $\mu \cdot \sigma^2$로 정규화하고, 동시에 **이동평균** (running_mean · running_var)을 조금씩 갱신한다

`model.eval()`

쌓아 둔 **이동평균**으로 정규화한다. 사진 1장이 와도, 배치 구성이 달라도 **같은 답**

```
# 평균 10 · 표준편차 2 인 배치를 200번 통과시킨 뒤
running_mean = 9.966   running_var = 4.116  # 진짜 10 · 4
eval 모드에서 14.0 → 1.988                  # (14 - 10) / 2 = 2
BatchNorm1d, train 모드, 1장 → ValueError # 분산을 잴 수 없다
```

uv run code/06_mechanics.py 의 [3] · 6차시 함정과 같은 스위치(model.eval)

배치 평균이 흔들리는 만큼, 같은 사진도 매번 조금 다르게 보인다

배치 크기	배치 평균의 흔들림 (표준편차)	이론값 1/√n
2	0.727	0.707
8	0.358	0.354
64	0.126	0.125

약한 규제 — 같이 묶인 사진에 따라 정규화가 달라진다. 약한 **노이즈 주입**과 같아 외우기가 조금 어려워진다

배치가 너무 작으면 흔들림이 커져 학습 자체가 불안해진다. 배치 2~4라면 GroupNorm 등 다른 정규화를 쓴다

uv run code/06_mechanics.py 의 [4] · 평균 0 · 표준편차 1 인 데이터, 배치 평균을 1,000번 잼

7차시 · ② 데이터 증강 — 무엇을 하나

정답이 바뀌지 않는 변형으로 사진을 늘린다

이번 실험

사방 4px 여백을 두고 **무작위로 잘라내기**
+ 50% 확률로 **좌우 뒤집기**

epoch마다 같은 사진이 조금씩 다르게 보인다 — 2,000장이 사실상 훨씬 많아진다

정답이 바뀌는 변형은 금지

숫자 6을 뒤집으면 9 · 좌회전 표지판을 좌우로 뒤집으면 우회전

“이 변형을 해도 사람이 같은 답을 하나?”로 고른다

3차시 “데이터로 막기”의 이미지판 — 모으지 않고 **늘린다**. 증강은 **학습 데이터에만** 건다.

code/07_images.py — transform(augment=True) · RandomCrop(32, padding=4) · RandomHorizontalFlip()

학습용 로더는 셋 — 증강 켠 것, 끈 것, 검증

```
steps = [transforms.RandomCrop(32, padding=4),
         transforms.RandomHorizontalFlip()]

train = CIFAR10(…, transform=증강 켬)   # 가중치를 배운다
plain = CIFAR10(…, transform=증강 끔)   # 학습 정확도를 잰다
val   = CIFAR10(test, transform=증강 끔) # 검증

# 학습 2,000장 = 클래스마다 시드로 섞어 200장
indices = subset_indices(train.targets)
```

왜 둘? 증강을 켠 채 학습 정확도를 재면 낮게 나와 **간격을 잘못 읽는다** — 같은 사진을 증강 없이 다시 잰다

작은 CNN: 합성곱 3층(32·64·128) + 완전연결 · 40 epoch · Adam 1e-3

code/07_images.py — transform(), loaders() 를 줄여 옮김 · “증강 켬/끔” 은 transform(augment=True/False)

7차시 · 작은 CNN — 결과

증강은 외우기를 막고, BN은 학습을 돕는다

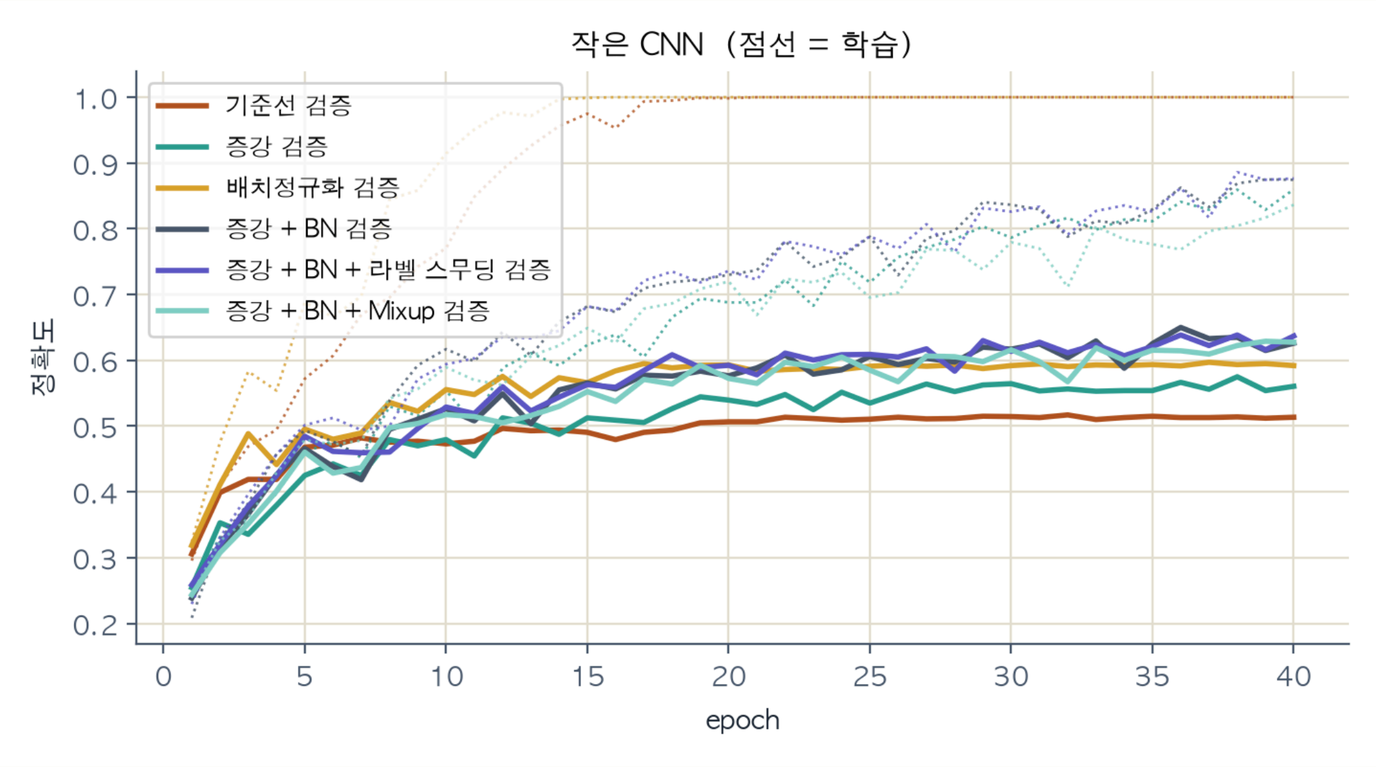
방법 (40 epoch)	학습	검증(마지막)	간격
기준선	1.000	0.514	0.487
증강	0.860	0.561	0.300
배치정규화	1.000	0.592	0.408
증강 + BN	0.875	**0.626**	0.249
증강 + BN + 라벨 스무딩	0.877	0.637	0.240
증강 + BN + Mixup	0.837	0.628	0.209

증강: 학습 100% → 86%, 간격 49 → 30%p. 외우기를 막았다

BN: 검증 +7.9%p, 그러나 학습은 여전히 **100%** — 막은 것이 아니라 **더 잘 배우게** 했다

uv run code/07_images.py 의 [A] · CIFAR-10 학습 2,000장 · 검증 2,000장(1장 = 0.05%p) · 시드 1개 · MPS

점선(학습)이 100%에 붙는 쪽과 못 붙는 쪽



실선 = 검증 · 점선 = 학습(증강 끈 같은 사진) — figs/07_small_cnn.png

정답을 덜 확신하게, 사진을 섞어서

라벨 스무딩 (ε = 0.1)

정답 칸 1.0 대신 **0.91**, 나머지 9칸에 **0.01**씩 — “100% 확신하지 마라”는 벌점

```
cross_entropy(logits, y,
  label_smoothing=0.1)
```

Mixup (α = 0.2)

두 사진을 λ : (1−λ)로 겹치고, **정답도 같은 비율**로 섞는다. λ ~ Beta(0.2, 0.2) — 대부분 0이나 1 근처

```
x = lam*x + (1-lam)*x[perm]
loss = lam*CE(y) + (1-lam)*CE(y[perm])
```

code/07_images.py — batch_loss(mode="smooth" | "mixup")

이 실험에서는 뚜렷한 차이가 없었다

증강 + BN 위에	검증 변화	간격
(그대로)	0.626	0.249
+ 라벨 스무딩	+1.1%p	0.240
+ Mixup	+0.2%p	**0.209**

검증 2,000장, 시드 1개 — **1%p 안팎은 흩어짐 안**입니다(2차시). 말할 수 있는 것: Mixup이 간격을 가장 많이 줄였다(학습 정확도를 끌어내렸다). 정확도 이득은 이 설정으로는 판정할 수 없다.

uv run code/07_images.py 의 [A] · 판정하려면 시드를 여러 개로 돌려 평균 ± 표준편차를 본다(실습 3번)

7차시 · ③ 전이학습 — 무엇을 하나

ImageNet 120만 장으로 배운 몸통을 빌리고, 머리만 새로 단다

처음부터

ResNet18을 무작위 가중치로 시작 · 2,000장으로 전부 배운다

몸통 동결

사전학습 몸통은 **그대로** · 마지막 fc(10칸)만 학습

전부 미세조정

몸통도 학습하되 **학습률 1e-4**로 살살 · 머리는 1e-3

몸통이 이미 아는 것(가장자리 · 질감 · 모양)은 2,000장으로 다시 배울 필요가 없다 — 배울 파라미터가 줄면 **외울 여지도 줄어든다**

code/07_images.py — resnet(pretrained, freeze), finetune_groups() · 셋 다 증강 켬 · 12 epoch · 사진은 96px로 키움

빌린 몸통의 습관을 지켜 준다

```
model = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(512, 10)                       # 새 머리 (1000칸 → 10칸)

stats = IMAGENET_STATS                              # ① 몸통이 배운 정규화 그대로
transforms.Resize(96)                               # ② 32px는 너무 작다
optimizer = Adam([{"params": body, "lr": 1e-4},     # ③ 몸통은 살살
                  {"params": head, "lr": 1e-3}])

# ④ 동결할 때: 매 epoch model.train() 뒤에
for m in model.modules():
    if isinstance(m, nn.BatchNorm2d) and 동결됨(m): m.eval()
```

code/07_images.py — resnet(), finetune_groups(), keep_frozen_bn() 를 줄여 옮김 · “동결됨” 은 requires_grad 가 모두 False

7차시 · ③ 전이학습 — 결과

같은 2,000장, 검증 44.5% → 82.6%

ResNet18 · 12 epoch	학습	검증(마지막)	간격
처음부터	0.535	0.445	0.090
몸통 동결	0.854	0.789	**0.066**
전부 미세조정	0.994	**0.826**	0.168

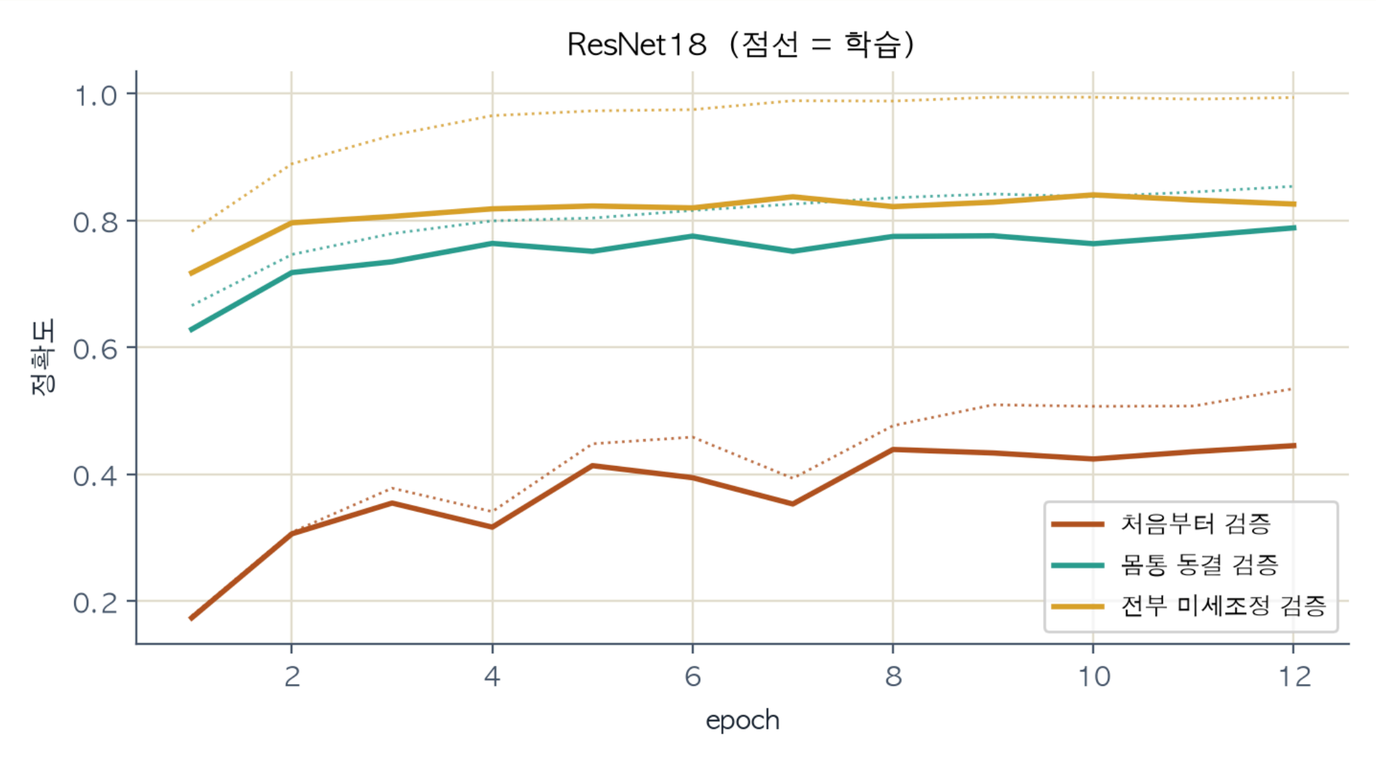
처음부터: 간격은 작지만 **학습도 53%** — 과적합이 아니라 아직 덜 배운 것 (1차시의 과소적합)

동결: 간격이 가장 작다. **미세조정**: 학습 99% · 간격은 크지만 검증이 가장 높다

uv run code/07_images.py 의 [B] · 작은 CNN 최고(증강 + BN)는 0.626 · 시드 1개

7차시 · ③ 전이학습 — 곡선으로 보기

빌린 몸통은 첫 epoch부터 앞서 있다



실선 = 검증 · 점선 = 학습(증강 끈 같은 사진) — figs/07_transfer.png

7차시 · 함정 ① 빌린 몸통 다루기

조용히 점수를 깎는 네 가지

실수	무슨 일이 생기나 → 대신
자기 데이터 평균·표준편차로 정규화	몸통이 배운 입력 분포와 어긋난다 → 사전학습 모델의 **ImageNet 통계**를 그대로
동결했는데 BN은 train 모드	running_mean/var가 덮어써져 몸통이 몰래 바뀐다 → 동결한 BN은 **eval()**
몸통도 1e-3 같은 큰 학습률	배운 것을 첫 몇 step에 망가뜨린다 → 몸통은 **10배 작게**
32px 그대로 넣기	224px로 배운 몸통엔 너무 작다 — 마지막 특징 지도가 1×1 → **키워서** 넣는다(여기선 96px)

⚠ 넷 다 **오류가 나지 않습니다**. 점수만 몇 %p 낮게 나와 “전이학습이 별로네”로 결론 나기 쉽습니다.

code/07_images.py — IMAGENET_STATS · keep_frozen_bn() · LR_BODY · RESNET_SIZE

2차시의 함정이 이미지에서 다시 나온다

1. **거의 같은 사진**(연사 · 같은 사람 · 같은 영상의 프레임)이 학습과 검증에 갈라 들어가면 누수 → 사람·영상 단위로 묶어 나눈다 (**GroupKFold**)

2. **검증에도 증강**을 걸면 잴 때마다 점수가 바뀐다 → 증강은 학습 로더에만

3. **증강된 사진으로 학습 정확도**를 재면 간격을 작게 읽는다 → 같은 사진을 증강 없이 다시 잰다

4. 곡선을 보고 고른 **검증 최고점**을 성능으로 보고 → 마지막 epoch, 또는 따로 둔 시험 데이터로

7차시 · 실습 20분

사진이 적을 때 무엇이 가장 효과적인가

1. PER_CLASS를 50 · 200 · 500으로. 전이학습의 이득(미세조정 – 처음부터)은 사진이 많을수록 커지나, 줄어드나?

2. EPOCHS_SMALL = 80. 증강 쪽 검증은 더 오르나? 기준선은?

3. SEED(common.py)를 0 · 1 · 2로 바꿔 “증강 + BN” vs “+ 라벨 스무딩”을 평균 ± 표준편차로. 차이가 흩어짐보다 큰가?

GPU(또는 Apple MPS)로 [A] 약 5분, [B] 약 10분. 조별로 나눠 돌리고 표를 모은다.

code/07_images.py 맨 위 상수 · SEED 는 code/common.py

O 일까 X 일까

Q1 이 실험에서 배치정규화는 학습 정확도를 낮춰 외우기를 막았다

Q2 손글씨 숫자 분류에 상하 뒤집기 증강을 쓰면 안전하다

Q3 requires_grad = False로 동결하면 BN의 이동평균도 그대로 유지된다

Q4 학습·검증 간격이 가장 작은 설정을 고르면 된다

7차시 정리

세 줄로

1 배치정규화는 학습을 **안정시키고**, 규제는 덤이다 — 평가는 **이동평균**으로(model.eval)

2 증강은 **정답이 안 바뀌는 변형**으로 외우기를 직접 막는다: 간격 49 → 30%p

3 사진이 적으면 **먼저 빌린다**: 같은 2,000장으로 62.6% → **82.6%**

다음 차시 — 지금까지 배운 것으로, 넘겨받은 모델을 진단하고 고친다

8차시 · 90분 · 종합

“96.7%” 모델을 넘겨받았다

① 숫자가 정직한가 → ② 과적합인가 → ③ 무엇으로 막나 → ④ 시험은 한 번

1~7차시의 도구를 순서대로 꺼내 쓴다

code/08_diagnose.py · 약 10초

8차시 · 상황

설비 고장 예측 — 전임자의 보고서

보고서 요약

"설비 1,200대의 센서 20종과 정비 기록으로 고장을 예측했습니다. 그래디언트 부스팅의 깊이를 비교해 가장 좋은 설정을 골랐고, **시험 정확도는 96.7%**입니다."

특징 21개 = 센서 20 + **"수리 기록 있음"** 1

고장 비율 51% · 라벨 8%는 무작위로 다시 뽑힘 (노이즈)

당신의 임무: 이 숫자를 **믿을지 말지** 정하고, 믿을 수 있는 숫자를 다시 만든다.

데이터는 make_classification 으로 만든 합성 데이터 — code/08_diagnose.py make_data()

8차시 · 진단 순서도

막기 전에 재고, 재기 전에 의심한다

① **정직한가**
누수 · 시험 재사용 · 미래 정보
— 2차시

→

② **과적합인가**
학습 곡선 · 간격과 추세
— 1·3차시

→

③ **무엇으로**
데이터 · 규제 · 크기 · 멈춤
— 3~7차시

→

④ **시험 한 번**
서랍의 시험지
— 2차시

```
# 맨 처음 — 시험지를 떼어 서랍에. 마지막에 딱 한 번만 연다
x_dev, x_test, y_dev, y_test = train_test_split(x, y, test_size=0.2, stratify=y)
```

8차시 · ① 숫자가 정직한가

미래의 열 하나를 빼자 96%가 85%로

무엇을 쟀나	정확도
전임자 — 시험으로 고르고 같은 시험으로 보고	0.967
“수리 기록 있음” **한 열만**으로 교차검증	0.964
개발 데이터 교차검증, “수리 기록” 포함	0.963
개발 데이터 교차검증, 예측 시점에 있는 열만	**0.849**

⚠ “수리 기록 있음”은 **고장이 난 뒤에야** 생기는 값입니다. 내일의 고장을 예측하는 오늘에는 없습니다. 정답을 거의 그대로 들고 있던 열(정답과 97% 일치)입니다.

uv run code/08_diagnose.py 의 [보고서] · [①]

8차시 · ② 과적합인가

학습 100%, 간격 15%p — 검증 증가는 둔화(+0.3%p, 흩어짐 1.2%p 안)

학습 수	학습	검증	간격
153	1.000	0.732	0.268
307	1.000	0.802	0.198
460	1.000	0.820	0.180
614	1.000	0.846	0.154
768	1.000	0.849	0.151

진단: 분산형 과적합 (3차시의 트리 쪽 곡선)

처방 후보: 모델을 묶는다(4·5차시). 더 모으기는 효과가 **불확실**

uv run code/08_diagnose.py 의 [②] 표 · 누수 열을 뺀 뒤 · 그래디언트 부스팅 기본 설정

8차시 · ③ 무엇으로 막나

후보를 교차검증으로 — 평균과 흩어짐을 함께

방법	교차검증 평균	± 표준편차
로지스틱 회귀 (단순 모델)	0.748	0.029
부스팅 기본 설정	0.849	0.012
조기 종료만	0.846	0.030
규제 탐색 최선 (12개 중 1등 — 낙관적)	0.855	0.022
폴드별 차이 (탐색 – 기본)	+0.006	0.013

단순 모델은 **10%p 낮다** → 이 문제엔 복잡한 모델이 필요하다

같은 폴드끼리 빼도 +0.6 ± 1.3%p — **흩어짐보다 작다**

uv run code/08_diagnose.py 의 [③] 표 · 고른 설정: max_depth=None · min_samples_leaf=5(기본 20보다 느슨) · l2=0

8차시 · ④ 시험은 한 번

서랍을 열었다 — 고른 규제가 1대 차이로 졌다

부스팅 기본 설정

0.875

교차검증으로 고른 규제

0.871

⚠ **여기서 하면 안 되는 일** — “그럼 기본 설정으로 바꿔서 다시 시험하자.” 시험 점수를 보고 고른 순간, 서랍의 시험지는 **검증셋이 됩니다**(2차시 ⑤).

시험지 240대 — 1대 = 0.42%p. 두 모델의 차이는 정확히 **설비 1대**입니다.

uv run code/08_diagnose.py 의 [④] 표 · 시험지는 맨 처음 떼어 둔 20%

8차시 · 다시 쓴 보고서

가장 큰 개선은 “정직하게 잰 것” 자체였다

“예측 시점에 알 수 있는 센서 20종만 쓰면 정확도는 **약 85~87%**입니다(교차검증 0.849 ± 0.012, 한 번 연 시험 0.875). 이전 보고의 96.7%는 고장 후에 생기는 ‘수리 기록’을 입력으로 썼고, 시험으로 설정을 고른 숫자였습니다.”

“규제 탐색은 폴드 간 흩어짐보다 큰 개선을 보이지 않아 **기본 설정을 유지**합니다. 데이터를 더 모으는 효과는 **불확실**합니다 (마지막 구간 증가 +0.3%p가 흩어짐 1.2%p 안).”

숫자는 내려갔지만, 이제 **현장에서 실제로 나올 숫자**입니다.

8차시 · 미니 프로젝트 25분

이번엔 여러분이 넘겨받는다

과제 — 강사가 나눠 준 `make_data()` 변형본(문제가 2~3개 심어져 있다)을 받아 08_diagnose.py의 네 단계로 진단

제출 — 앞 슬라이드 같은 **보고서 두 문단** + 근거 표

채점 기준	배점
시험지를 맨 처음 뗐나	20
심어진 문제를 찾았나	30
과적합 진단 근거가 곡선인가	20
비교에 흩어짐을 함께 썼나	20
시험을 한 번만 열었나	10

심을 수 있는 문제 예: 미래 정보 열 · ID 열 · 같은 설비의 중복 행(그룹 누수) · 시간 순서 · 라벨 노이즈

부록 A 전체 · 한 장 요약

과적합 점검표

차시	확인할 것	이렇게
2	시험지를 맨 처음 뗐나	train_test_split(…, stratify=y) 후 서랍에
2·8	예측 시점에 없는 정보가 섞였나	열마다 “그때 알 수 있나?” · 단일 열 성능 확인
2	전처리가 파이프라인 안인가	make_pipeline(…) · 라벨 셔플 검사(찍기 수준이 나와야)
1·3	과적합인가 과소적합인가	학습·검증 곡선의 간격과 추세
3	더 모으면 나아지나	learning_curve — 아직 오르나
4·5	모델을 얼마나 묶나	validation_curve로 α·깊이·잎 크기
6·7	신경망은 언제 멈추나	검증 손실 + 인내심 · 가장 좋았던 가중치로
2·8	차이가 진짜인가	평균 ± 표준편차 · 흩어짐보다 큰가

8차시 · 확인 퀴즈

O 일까 X 일까

Q1 교차검증을 정직하게 했다면, 입력 열 중에 미래 정보가 있어도 숫자는 부풀지 않는다

Q2 교차검증 평균이 0.6%p 높고 표준편차가 2%p라면, 개선이라고 단정하기 어렵다

Q3 시험 점수가 기대보다 낮으면 설정을 바꿔 같은 시험지로 다시 재 보는 것이 좋다

Q4 이번 사례에서 가장 큰 점수 변화를 만든 것은 규제가 아니라 누수 제거였다

8차시 정리

세 줄로

1 순서: **정직한가 → 과적합인가 → 무엇으로 → 시험 한 번**

2 흩어짐보다 작은 차이는 **개선이라 부르지 않는다**

3 낮아도 정직한 숫자가 **현장에서 신뢰**를 얻는다

부록 A를 마치며

학습 점수는 외운 양,
검증 점수는 배운 양.
그리고 그 검증 점수부터 의심한다.

코드 10개 · 그림 · 이 슬라이드: 콘텐츠 저장소 deeplearning/appendix-overfit/